



A

Report on

**Prakash: Smart Streetlight Maintenance and Decision
Support System**

Prepared For

Natural Language Processing (NLP)

Complex Engineering Problem (CEP) · SDG 11: Sustainable Cities and Communities

Team 6

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,

NAVI MUMBAI 400706

ACADEMIC YEAR 2025-26



A

Report on

**Prakash: Smart Streetlight Maintenance and Decision Support
System**

Prepared For

Natural Language Processing (NLP) · CEP

Submitted by: Team 6

Bharath Rathinasapabathy 123A8012

Gregory 123A8020

Harshwardhan Ahire 123A8004

Department of Artificial Intelligence & Data Science

Under the guidance of

Dr. Vidya Keshwani

SIES GRADUATE SCHOOL OF TECHNOLOGY NERUL,

NAVI MUMBAI 400706

ACADEMIC YEAR 2025-26

Acknowledgement

We, Team 6, express our sincere gratitude to our guide, **Dr. Vidya Keshwani**, for the direction, feedback and encouragement she gave us throughout this project. Her suggestion to treat the work as a Complex Engineering Problem aligned with a Sustainable Development Goal pushed us to build a complete, usable system instead of a classroom prototype.

We thank the Department of Artificial Intelligence & Data Science and SIES Graduate School of Technology, Nerul, for the laboratory facilities and the academic environment that made this work possible. We also thank our classmates, whose questions during the interim presentation helped us sharpen the problem statement and the evaluation.

Finally, we acknowledge the open-source communities behind Python, scikit-learn, Flask, SQLite, Leaflet and Chart.js. Their tools allowed three students to build and test a full-stack, multilingual AI system on ordinary laptops.

Team 6

Abstract

Faulty streetlights reported by citizens are still handled manually in most Indian cities. A clerk reads each phone call, WhatsApp message, email or tweet, decides how urgent it is, tries to work out which pole is meant, and calls a technician. The result is delayed repairs, inefficient crew scheduling and unsafe dark roads. This report presents **Prakash**, an AI-driven streetlight maintenance and decision support system that automates this journey end to end.

Prakash accepts complaints from six channels (WhatsApp, SMS, email, X/Twitter, call-centre transcripts and a web portal) and processes every message with one Natural Language Processing pipeline: channel-specific text clean-up, language identification for English, Hindi, Marathi and Hinglish, fault classification into eight classes using TF-IDF word and character n-grams with logistic regression, named entity recognition for pole IDs, landmarks, roads, sectors, durations and fault phrases, and an explainable severity score from 0 to 100. The extracted location is geocoded, snapped to the nearest lamp in a GIS lamp register and checked against open tickets so that duplicate reports merge instead of creating extra work. Messages the system is unsure about are sent to a human review queue, and the operator's corrections are used to retrain the classifier.

On top of the NLP layer, Prakash identifies maintenance hotspots with DBSCAN clustering, predicts the probability that each lamp fails within 14 days with a gradient-boosted model, forecasts weekly fault counts with damped Holt smoothing, and builds crew routes with a priority-, skill- and shift-aware insertion heuristic improved by 2-opt and or-opt moves. A role-based web application (citizen, technician, admin) with live updates over server-sent events, alerts, a technician route view and a citizen tracking portal puts the results into use. The backend is written in Python with Flask and SQLite; the interface uses Leaflet and Chart.js.

On a held-out test of 480 paraphrased complaints that never appear in training, the classifier reaches **87.5% accuracy (macro-F1 0.875)** against 42.7% for the keyword-rule baseline of our first prototype, and 91.7% on 24 hand-written messages. The failure-risk model reaches an AUC of 0.80, and on the demonstration data the route optimiser cuts total travel by about one third and reduces the average time to reach Critical and High jobs from 105 to 12 minutes. Because no public labelled streetlight-complaint corpus exists, the training complaints, lamp network and history are synthetic; the report states this limitation openly and describes how the system retrains on real data. The project supports **SDG 11**, in particular target 11.7 (safe public spaces), target 11.2 (safe transport systems) and target 11.3 (participatory urban planning).

Keywords: Natural Language Processing, text classification, named entity recognition, code-mixed text, GIS, DBSCAN, predictive maintenance, route optimisation, human-in-the-loop, SDG 11.

Table of Contents

Acknowledgement	1
Abstract	2
CHAPTER 1 Introduction.....	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Motivation.....	7
1.4 Objectives	7
1.5 Alignment with SDG 11	7
1.6 Scope and Limitations.....	8
1.7 Organisation of the Report.....	8
CHAPTER 2 Literature Review and Technical Background	9
2.1 Citizen Complaint Systems.....	9
2.2 Text Classification	9
2.3 Named Entity Recognition and Geoparsing	9
2.4 Code-Mixed and Multilingual Text	10
2.5 Spatial Clustering for Hotspots	10
2.6 Predictive Maintenance and Forecasting	10
2.7 Vehicle Routing and Scheduling	10
2.8 Research Gap and Contribution	11
CHAPTER 3 Requirements and Problem Analysis.....	12
3.1 Stakeholders and Roles	12
3.2 Functional Requirements	12
3.3 Non-Functional Requirements	13
3.4 Use Cases	13
3.5 Why This Is a Complex Engineering Problem	14
CHAPTER 4 System Design	15
4.1 Architecture Overview.....	15
4.2 Layers and Responsibilities	15
4.3 Data Model.....	16
4.4 Ticket Life Cycle	16
4.5 API Design.....	17
4.6 Real-Time Updates	17
4.7 Security Design.....	17
4.8 Synthetic Data Generation	18
CHAPTER 5 NLP Methodology	19
5.1 Training and Test Data.....	19

5.2 Text Normalisation	19
5.3 Language Identification	20
5.4 Fault Classification	20
5.5 Named Entity Recognition.....	21
5.6 Severity Scoring.....	21
5.7 Geocoding and Lamp Matching.....	22
5.8 Duplicate Detection	23
5.9 Human-in-the-Loop Review	23
5.10 Automatic Multilingual Reply	23
5.11 Worked Example	23
CHAPTER 6 Predictive Analytics and Optimisation	25
6.1 Hotspot Detection with DBSCAN	25
6.2 Lamp Failure-Risk Model.....	25
6.3 Weekly Fault Forecast	25
6.4 Priority Score	26
6.5 Technician Scheduling.....	26
6.6 Alerts Engine	26
CHAPTER 7 Implementation.....	27
7.1 Tools and Technologies	27
7.2 Project Structure.....	27
7.3 User Interface.....	28
CHAPTER 8 Testing and Results.....	40
8.1 Test Strategy	40
8.2 Classifier Results	40
8.3 Effect of the Safety Rule and Review Threshold.....	41
8.4 Failure-Risk Model Results	41
8.5 Scheduling Results.....	42
8.6 Performance	42
8.7 Threats to Validity and Limitations	43
CHAPTER 9 Social Impact, Ethics and Sustainability	44
9.1 Expected Impact.....	44
9.2 Ethical Considerations	44
9.3 Sustainability.....	44
CHAPTER 10 Mapping to Course Outcomes, Programme Outcomes and Knowledge Profiles	45
10.1 Course Outcome Mapping	45
10.2 Programme Outcome Mapping.....	45
10.3 Knowledge Profile (WK) Mapping	46

CHAPTER 11 Conclusion and Future Scope	48
11.1 Conclusion	48
11.2 Future Scope	48
References	49
Appendix A: Running the Project.....	51
Appendix B: REST API Summary	52
Appendix C: Sample Complaints and Outputs	53

CHAPTER 1

Introduction

1.1 Background

Streetlights are among the most visible services a city provides, and among the most noticed when they fail. A dark road raises the risk of road accidents, theft and harassment, and it discourages people, especially women, children and older residents, from using public space after sunset. Yet maintaining a large network of lamps is a logistics problem. A mid-sized municipal ward can have tens of thousands of lamps, hundreds of complaints a week, and a handful of crews. Each complaint arrives in free text, in whichever language and through whichever channel the citizen finds convenient.

In practice, the work flow is largely manual. A complaint is read by a clerk, entered in a register, and handed to a supervisor who decides which crew goes where. Locations are described in words such as "near the school gate" or "opposite the temple" rather than coordinates, so crews waste time finding the right pole. Duplicate complaints about the same lamp create duplicate work orders, while a hanging live wire may wait in the same queue as a lamp that merely flickers. Crews are assigned by arrival order rather than by priority, certification or geography, so they criss-cross the city.

Recent advances in Natural Language Processing, geographic information systems (GIS) and machine learning make it possible to automate most of these decisions while keeping a human in control of the uncertain ones. This project, **Prakash** (Sanskrit and Hindi for "light"), explores how far a complete pipeline can go with techniques that students can implement, explain and test.

1.2 Problem Statement

Faulty streetlights reported by citizens are often processed manually, leading to delayed repairs, inefficient maintenance scheduling, and increased public safety concerns. Develop an AI-driven Smart Streetlight Maintenance and Decision Support System that integrates Natural Language Processing (NLP), Machine Learning, Geographic Information Systems (GIS) and predictive analytics to collect maintenance requests from multiple communication channels, automatically classify fault types, extract streetlight locations using Named Entity Recognition (NER), assess fault severity, identify maintenance hotspots, predict recurring failures based on historical data, optimise technician scheduling and resource allocation, and provide real-time dashboards with alerts to support efficient streetlight maintenance and improve public safety.

1.3 Motivation

- **Safety.** Hazards such as exposed wires and leaning poles must be identified in minutes, not days.
- **Language diversity.** Real complaints in Navi Mumbai arrive in English, Hindi, Marathi and in Hinglish (Hindi written in Roman script). A system limited to formal English would miss a large share of citizens.
- **Unclear locations.** A complaint is only useful if a crew can find the pole. Entity extraction and a lamp register turn words into coordinates.
- **Wasted effort.** Duplicate tickets, unprioritised queues and unplanned routes waste crew hours and fuel.
- **Accountability.** Citizens and auditors need to see why a ticket was rated critical and who fixed it and when.

1.4 Objectives

1. Collect maintenance requests from multiple channels (WhatsApp, SMS, email, X/Twitter, call-centre transcripts, web portal, bulk upload) into a single queue.
2. Classify the fault type automatically across English, Hindi, Marathi and Hinglish text.
3. Extract pole IDs, landmarks, roads, sectors, durations and fault phrases using Named Entity Recognition, and convert them to a location on the map.
4. Score severity from 0 to 100 with a visible, auditable explanation.
5. Identify maintenance hotspots and recurring failure locations using spatial clustering.
6. Predict which lamps are likely to fail next and forecast weekly fault volumes from historical data.
7. Optimise technician scheduling under priority, skill-certification and shift-length constraints.
8. Provide real-time dashboards and alerts for the control room, a route view for technicians and a tracking portal for citizens.
9. Send uncertain cases to a human reviewer and learn from the corrections.

1.5 Alignment with SDG 11

Sustainable Development Goal 11 aims to "make cities and human settlements inclusive, safe, resilient and sustainable" [1]. Prakash contributes to three of its targets:

Table 1: SDG 11 targets addressed by the project

Target	Statement (summary)	Contribution of Prakash
11.7	Provide universal access to safe, inclusive and accessible public spaces, in particular for women, children, older persons and persons with disabilities.	Faster repair of dark roads, priority points for women, students and elderly users in the severity score, public city map of lamp status.
11.2	Provide access to safe, affordable, accessible and sustainable transport systems and improve road safety.	Hazard detection (leaning poles, exposed wires) goes to certified crews first; faster restoration of lighting on roads and junctions.
11.3	Enhance inclusive and sustainable urbanization and capacity for participatory planning and management.	Citizens report in their own language, confirm existing faults, track progress and rate repairs; the city gets data on where infrastructure fails.

The project also supports SDG 9 (resilient infrastructure) through predictive maintenance, and SDG 7 (affordable and clean energy, target 7.3 on energy efficiency) because lamps that burn in daytime are detected as a separate fault class and cleared.

1.6 Scope and Limitations

In scope: multilingual complaint understanding, lamp matching, duplicate merging, hotspot detection, failure-risk prediction, weekly forecasting, skill-aware crew scheduling, role-based web application with live dashboard, alerts, citizen tracking and technician route.

Out of scope: live GPS tracking of crews, integration with a real municipal ticketing system or WhatsApp Business account, IoT sensors inside lamps, and production-scale deployment. Complaint text used for training, the lamp register and twelve weeks of maintenance history are **synthetic**, generated for this project, because no public labelled corpus of streetlight complaints exists. All reported accuracy figures therefore describe the pipeline on synthetic data and should not be read as real-world performance.

1.7 Organisation of the Report

Chapter 2 reviews related work and the techniques we build on. Chapter 3 analyses requirements and shows why the problem is a Complex Engineering Problem. Chapter 4 presents the system design. Chapters 5 and 6 describe the NLP pipeline and the predictive and optimisation modules in detail. Chapter 7 describes the implementation and the user interface. Chapter 8 reports testing and results. Chapter 9 discusses social impact, ethics and sustainability. Chapter 10 maps the work to course outcomes, programme outcomes and knowledge profiles. Chapter 11 concludes with future scope.

CHAPTER 2

Literature Review and Technical Background

2.1 Citizen Complaint Systems

Several civic reporting platforms already let residents report faulty infrastructure. FixMyStreet (mySociety) and SeeClickFix are widely used examples, and many municipalities run their own 311-style helplines and mobile apps. Their strength is easy intake and public visibility of reports. Most depend on structured form fields (category, pin on a map) filled in by the citizen, or on staff who read and categorise free text. Open 311 data sets such as the New York City 311 Service Requests [18] and the City of Chicago 311 Service Requests [19] contain street-light complaints and show how large the volumes are, but they are English-only and structured. Free-text, multilingual and code-mixed reports through chat channels are much less served, and that is the gap this project targets.

2.2 Text Classification

Classifying short complaints into a small set of fault types is a supervised text-classification task. A standard and strong baseline represents each message as a vector of **TF-IDF** weights [2] and trains a linear classifier such as logistic regression [20]. Term frequency rewards words that appear often in a message, and inverse document frequency down-weights words that appear in almost every message. Character n-gram features extend the idea to sub-word units, which makes the representation robust to spelling variation and to transliteration, both common in Indian-language chat messages.

Pre-trained transformer models such as BERT [10] and its smaller variant DistilBERT [11] give higher accuracy when enough labelled data and compute are available, and multilingual models such as MuRIL [12] are trained specifically on Indian languages and their transliterated forms. We chose the TF-IDF and logistic-regression route for this project because there is no labelled real data, training must finish in seconds on a laptop, and a linear model is transparent: the weight of each word in each decision can be shown to the operator. The pipeline is modular so a transformer can replace the classifier later.

2.3 Named Entity Recognition and Geoparsing

Named Entity Recognition (NER) finds and labels spans of text that refer to entities such as places or durations [4]. Approaches range from hand-written rules and gazetteers (lists of known names) to statistical sequence models and neural taggers. For complaints, the useful entities are domain-specific: pole identifiers, landmarks, roads, sectors, durations and fault phrases. With no annotated data, a hybrid of regular expressions and a gazetteer of local

landmarks is the practical choice, and it is easy to extend. Geoparsing, the step of turning a recognised place name into coordinates, is done by gazetteer look-up; coordinates are then matched to the nearest asset using great-circle distance computed with the haversine formula [22].

2.4 Code-Mixed and Multilingual Text

Messages that mix Hindi and English, either in Devanagari or in Roman script ("Hinglish"), are routine in India. They break word-level tokenisers trained on a single language and produce many unseen spellings. Character n-grams, script detection, and training data that deliberately includes the mixed forms are lightweight ways to handle them. The system detects the language so that its automatic reply can be written in the citizen's own language.

2.5 Spatial Clustering for Hotspots

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [5] groups points that have at least a minimum number of neighbours within a given radius, and labels isolated points as noise. It needs no preset number of clusters and can find clusters of any shape, which suits faults that line up along a road because they share a feeder or a cable. Using the haversine metric makes the radius a real distance in metres.

2.6 Predictive Maintenance and Forecasting

Predictive maintenance estimates the probability that an asset fails within a horizon, using features such as age, past failures and operating conditions. Gradient-boosted decision trees [6] are a strong, widely used model for such tabular data and provide feature importance for explanation. For counts over time, exponential smoothing with a damped trend [7] is a simple, robust forecaster that works with short histories, and it supports prediction intervals from the spread of past errors.

2.7 Vehicle Routing and Scheduling

Assigning jobs to crews and ordering the visits is a Vehicle Routing Problem (VRP) with additional constraints [9]. Exact methods do not scale well, so practical systems use heuristics. Cheapest insertion builds routes by adding each job at the position that increases cost least, and local-search moves such as 2-opt [8] (reverse a segment of the route) and or-opt (move a job) improve them. Skill constraints (only certified crews handle live-wire hazards) and shift length are modelled as hard limits.

2.8 Research Gap and Contribution

Table 2: Gaps addressed by this work

Gap in existing practice	How Prakash addresses it
Reports need structured forms or manual reading	One NLP pipeline reads free text from six channels.
English-only processing	English, Hindi, Marathi and Hinglish handled by the same model and answered in the citizen's language.
Location lost in words	NER plus gazetteer, pole-ID look-up and GPS pins resolve a message to a specific lamp.
Duplicates and flat queues	Reports about one lamp merge and raise its priority; severity is explainable.
Reactive maintenance only	Failure-risk prediction and hotspot detection support preventive work orders.
Crews dispatched by arrival order	Skill-, priority- and distance-aware optimiser compared with a manual baseline.
AI decisions are opaque	Every decision shows the words, rules and points behind it; low confidence goes to a human, and corrections retrain the model.

CHAPTER 3

Requirements and Problem Analysis

3.1 Stakeholders and Roles

Table 3: Stakeholders and their needs

Role	Needs	What the system gives them
Citizen	Report a fault easily in their own language; know that it was received; know when it will be fixed.	Web portal with live AI preview, photo and GPS pin, voice input, duplicate warning, status timeline, notifications, rating.
Technician	A clear list of jobs in a sensible order, with location and parts.	Optimised route on a map, start and finish buttons with notes and parts, shift toggle, history.
Control-room admin	See what is happening now, decide what to do, and trust the AI.	Live dashboard, ticket drawer with explanation, review queue, alerts, scheduling, analytics, model metrics, user management, audit log.
City administration	Evidence of service quality and where infrastructure fails.	SLA compliance, resolution times, hotspots, forecasts, CSV export.

3.2 Functional Requirements

Table 4: Functional requirements

ID	Requirement	Module
FR1	Accept complaints from WhatsApp, SMS, email, X/Twitter, call transcripts, web form and CSV import through a common ingest pipeline.	Ingest, webhooks
FR2	Clean channel noise (signatures, quoted replies, hashtags, mentions, emojis, speech fillers).	NLP
FR3	Detect language (English, Hindi, Marathi, Hinglish).	NLP
FR4	Classify fault type into eight classes with a probability for each.	NLP
FR5	Extract pole IDs, landmarks, roads, sectors, durations and fault phrases.	NLP (NER)
FR6	Score severity from 0 to 100 and list the reason for each point.	NLP
FR7	Geocode the location, snap to the nearest lamp, and merge duplicate reports.	GIS, services
FR8	Send low-confidence or location-less tickets to a human review queue and store corrections.	Review, learning

ID	Requirement	Module
FR9	Cluster faulty lamps into hotspots and find 90-day repeat-failure zones.	Analytics
FR10	Predict 14-day failure risk per lamp, with a rainfall what-if, and forecast weekly faults.	Analytics
FR11	Build skill-aware, priority-aware routes; dispatch on one click; auto-dispatch critical faults.	Scheduler
FR12	Show real-time dashboard, alerts, notifications and a citizen tracking timeline.	Frontend, SSE
FR13	Role-based access for citizen, technician and admin; audit log of administrative actions.	Security
FR14	Retrain the classifier from stored human corrections and report before/after accuracy.	Models

3.3 Non-Functional Requirements

- **Usability:** works on desktop and phone, light and dark themes, plain-language messages.
- **Performance:** live AI preview must respond in well under a second so it can run while the citizen types (measured mean 29 ms, Chapter 8).
- **Explainability:** each automated decision shows its evidence.
- **Security and privacy:** server-side role checks, hashed passwords, rate-limited login, escaped output, restricted uploads, minimal personal data.
- **Reliability:** a single command starts the system; the system works offline except for map background tiles.
- **Maintainability:** modular code (NLP, analytics, scheduler, services, API) with automated tests.

3.4 Use Cases

Table 5: Use cases by actor

Actor	Main use cases
Citizen	Register and sign in · report a fault · see live AI understanding · pin a location or use GPS · attach a photo · speak the complaint · confirm an existing fault ("me too") · track a ticket · rate a repair · view the city lamp map · read notifications.
Technician	Sign in · toggle shift · view optimised route · navigate to a job · start work · mark fixed with notes and parts · view history and rating.

Actor	Main use cases
Admin	Monitor dashboard · open a ticket · assign, correct, annotate, resolve or reject · review uncertain tickets · simulate channels · inspect hotspots and predictions · run and dispatch the optimiser · manage technicians and users · adjust SLA and thresholds · retrain the model · export data.

3.5 Why This Is a Complex Engineering Problem

The project was framed as a Complex Engineering Problem (CEP). the table below relates it to the usual attributes of such problems.

Table 6: Complex engineering problem attributes

Attribute	Evidence in this project
Depth of knowledge required	Needs NLP (representation, classification, NER), spatial analysis, machine learning, optimisation and web engineering together.
Conflicting requirements	Fast triage versus accuracy (human review threshold); urgent jobs first versus shortest travel; automation versus citizen trust and privacy.
Depth of analysis	No standard solution exists: model choice, features, severity design, dedupe rules and routing cost all needed analysis and experiments.
Familiarity of issues	Code-mixed Hinglish, vague locations and missing data are not covered by textbook NLP exercises.
Stakeholder involvement	Citizens, technicians, control-room staff and city administration have different, sometimes competing, needs.
Interdependence	Classification, location, severity, duplicates, hotspots, risk and scheduling feed each other; an error early in the pipeline propagates, which is why a review loop exists.

CHAPTER 4

System Design

4.1 Architecture Overview

Prakash follows a five-stage pipeline: **Collect, Understand, Locate, Predict and Decide, Act**. A Flask backend implements the stages as services over an SQLite database and exposes them through a REST API and a server-sent event stream. A single-page browser application renders three role-specific interfaces.

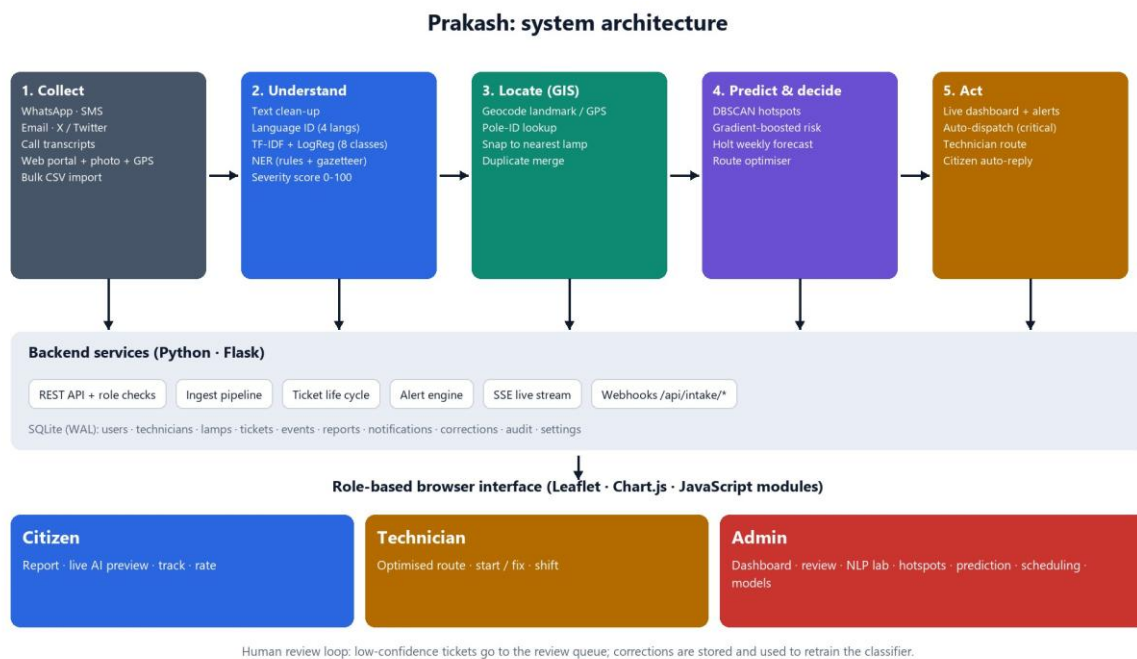


Figure 1: System architecture of Prakash

4.2 Layers and Responsibilities

Table 7: Layers of the system

Layer	Technology	Responsibility
Presentation	JavaScript modules, Leaflet, Chart.js, CSS	Role-based single-page app, maps, charts, live updates, responsive layout, themes.
API	Flask, signed-cookie sessions	REST endpoints, role checks, input validation, uploads, webhooks, SSE stream.
Services	Python modules	Ingest pipeline, duplicate handling, ticket life cycle, alerts, dashboard numbers, dispatch.
Intelligence	scikit-learn, NumPy	NLP pipeline, classifier, NER, severity, risk model, forecast, DBSCAN, scheduler.
Data	SQLite (WAL mode)	Users, technicians, lamps, tickets, events, reports, notifications, corrections, audit log, settings.

4.3 Data Model

Table 8: Main database tables

Table	Purpose and key fields
users	Account, role (citizen, technician, admin), password hash, active flag, last login.
technicians	Linked user, skills (HV-certified, bucket truck), base location, colour, shift status, capacity in minutes.
lamps	Lamp ID (for example SL-NER-021), coordinates, zone, age, supply-voltage variation, driver type, burning hours, pole condition, status.
tickets	Text, cleaned text, language, class, confidence, severity and reasons, entities, probabilities, location and lamp, status, SLA deadline, reporter, report count, technician, route order, ETA, resolution note, parts, rating.
ticket_events	Timeline of everything that happened to a ticket (who, what, when).
ticket_reports	Additional citizen reports merged into a ticket.
notifications	In-app messages for citizens and technicians.
corrections	Human-corrected labels, used to retrain the classifier.
alert_acks, audit, settings	Acknowledged alerts, administrative audit trail, configurable SLA hours and thresholds.

4.4 Ticket Life Cycle

A ticket enters as *Received* and is placed either in *Open* (confident classification and a usable location) or in *Review* (uncertain). An admin can release a reviewed ticket to Open, or reject it. Open tickets become *Assigned* when a technician is chosen, *In progress* when the technician starts, and *Resolved* when the job is fixed. Critical tickets are auto-dispatched to the nearest qualified technician. Duplicate reports never create a new ticket: they join the existing one and raise its priority.

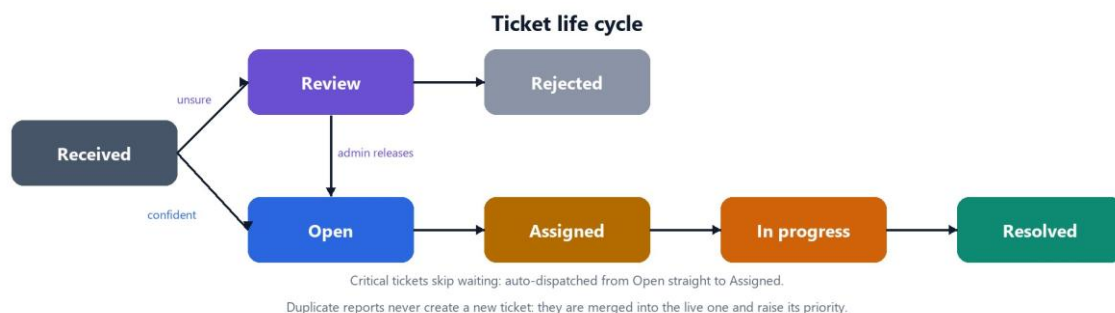


Figure 2: Ticket life cycle

4.5 API Design

The interface between browser and server is a set of JSON REST endpoints [13], grouped by role. A summary is given below; the complete list is in Appendix B.

Table 9: REST API groups

Group	Examples
Authentication	POST /api/auth/login, /register, /logout; GET /api/auth/me.
Citizen	POST /api/analyze (live AI preview), POST /api/tickets, GET /api/my/tickets, GET /api/nearby, POST /api/tickets/{id}/confirm and /rate, GET /api/city/map.
Technician	GET /api/tech/jobs, POST /api/tech/shift, POST /api/tech/jobs/{id}/start and /complete.
Admin	GET /api/admin/overview, /tickets, /hotspots, /predictions, /schedule, /analytics, /model, /alerts; POST /api/admin/schedule/dispatch, /model/retrain, /preventive.
Channels	POST /api/intake/{whatsapp, sms, email, twitter, call, web}, protected by an X-API-Key header or an admin session.
Live updates	GET /api/stream (server-sent events).

4.6 Real-Time Updates

Updates flow only from server to browser, so we use Server-Sent Events instead of WebSockets. When a ticket is created or changed, the server publishes an event. Admin connections receive every event with full detail; a citizen or technician receives only events about their own tickets, with reduced detail. The browser updates the feed, map, counters and charts without refreshing, shows red alerts for critical faults and raises notifications.

4.7 Security Design

- **Authentication:** passwords stored as salted hashes (Werkzeug); signed, HttpOnly, SameSite cookies; login attempts rate-limited.
- **Authorisation:** every API route checks the role on the server. A citizen sees their own tickets in full and only a public summary of others; a technician sees only assigned jobs. Automated tests verify these rules.
- **Input handling:** length limits, coordinates restricted to the service area, uploads limited to images up to a size limit and stored under random names, all output escaped before display.
- **Webhooks:** external channel calls need a shared API key.

- **Auditability:** administrative actions are written to an audit log; each ticket keeps its own event timeline.

4.8 Synthetic Data Generation

Because real data was not available, the project generates its own, in three parts. (1) **Lamp network:** 508 lamps in eight zones of Navi Mumbai (Airoli, Ghansoli, Vashi, Sanpada, Nerul, Seawoods, Belapur, Kharghar), laid out along road corridors and short roads through 12 landmarks and every railway station, using approximate real coordinates. Each lamp has age, supply-voltage variation, driver type, burning hours and pole condition. (2) **History:** twelve weeks of resolved tickets whose volume, fault mix and repair times follow realistic patterns; lamps that are older or have worse supply fail more often, so repeat offenders exist. All of it is generated through the real NLP pipeline, so the data is consistent with the models. (3) **Complaint text:** phrase banks in English, Hinglish, Hindi and Marathi combined with places, openers, closers, durations and random typos. Separate phrase banks are used for training and testing (Chapter 5).

CHAPTER 5

NLP Methodology

Every complaint, from any channel, passes through the same sequence of steps. This chapter describes each step, the algorithm behind it, and the design decisions.

5.1 Training and Test Data

The classifier distinguishes eight fault classes (listed in the table below). Complaints are composed from phrase banks, where each class has several phrasings in English, Hinglish, Hindi and Marathi. A complaint is built as *opener* + *fault phrase* + *place* + *duration* + *closer*, with random typos in about one fifth of messages. The **training** set contains 170 messages per class (1,360 in total). The **test** set contains 60 messages per class (480 in total) built from *different* phrasings that never occur in training, so that the score measures generalisation rather than memorisation. A third set of **24 hand-written, realistic messages** (including examples from our first prototype) provides a sanity check. In addition, a keyword-rule classifier like the one in our first prototype is kept as a baseline.

Table 10: Fault classes

Class	Meaning	Typical repair	Crew skills needed
outage	Lamp not working	30 min	standard
flicker	Flickering or intermittent	25 min	standard
dayburn	Burning in daytime (timer or photocell)	20 min	standard
hazard	Electrical or pole hazard (sparks, hanging wire, leaning pole)	60 min	HV-certified
dim	Dim or weak light	25 min	standard
vandal	Damaged or stolen fixture or cable	45 min	standard
timer	Controller or timer fault	35 min	standard
cluster	Multiple-light outage on a stretch	75 min	HV + bucket truck

5.2 Text Normalisation

Each channel has its own kind of noise, so the first step cleans the text before any model sees it. The table below lists the operations; the list of fixes applied to a message is stored with the ticket and shown to the operator.

Table 11: Text normalisation by channel

Channel	Clean-up applied
Email	Remove header lines (From, To, Date), merge the subject into the body, drop quoted replies (lines beginning with ">") and everything after a signature marker ("--", "Regards", "Sent from my").
X / Twitter	Remove retweet markers and @mentions; split hashtags written in CamelCase into words (#NerulStreetlight becomes "Nerul Streetlight").
Call centre	Remove speech fillers such as "uh", "um", "you know".
All channels	Remove URLs and emojis, expand SMS shorthand (plz, pls, stn, nr, opp), collapse whitespace.

5.3 Language Identification

If a message contains Devanagari characters it is Hindi or Marathi, and the two are separated by counting marker words typical of each language (for example आहे, नाही, जवळ for Marathi and है, नहीं, के पास for Hindi). A message in Roman script that contains two or more common romanised Hindi words (hai, nahi, band, andhera, batti, paas and similar) is labelled Hinglish; otherwise it is English. The label drives the language of the automatic reply.

5.4 Fault Classification

Representation

The cleaned text is converted to a sparse vector by concatenating two TF-IDF representations: word unigrams and bigrams, and character n-grams of length 2 to 5 within word boundaries. Sub-linear term frequency is used. For a term t in a document d of a corpus of N documents, the weight is

$$tfidf(t, d) = (1 + \log tf(t, d)) \times \log((1 + N) / (1 + df(t))) + 1$$

and each vector is length-normalised. Character n-grams keep similar spellings close (batti and bati, flickring and flickering) and give useful features for Devanagari, which has no spaces inside words that a simple tokeniser can exploit.

Classifier

A multinomial logistic regression model (regularisation strength $C = 12$) maps the vector $\mathbf{x}$ to class probabilities with the softmax function:

$$P(y = k | \mathbf{x}) = \exp(w_k \cdot \mathbf{x} + b_k) / \sum_j \exp(w_j \cdot \mathbf{x} + b_j)$$

The class with the highest probability is the prediction, and the full probability list is shown to the operator.

Explainability

For the predicted class k , the contribution of each word feature is its TF-IDF value multiplied by the corresponding weight w_k . The six largest positive contributions are shown as "words that drove this decision". For example, "flickering" and "on and off" drive the *flicker* decision, and "sparks" and "hanging from" drive *hazard*.

Safety rule

Missing an electrical hazard is far more costly than a false alarm. A deterministic safety rule therefore raises the probability of *hazard* to at least 0.55 whenever danger words (spark, live wire, exposed wire, electrocution, shock, short circuit, smoke, dangling, and Hindi and Marathi equivalents) appear, even if the statistical model preferred another class. This is reported in the interface as "safety rule applied".

5.5 Named Entity Recognition

The NER module combines regular expressions with a gazetteer of local places. It tags five entity types (see the table below). Overlapping matches are resolved by keeping the earliest and longest span, so "SIES College Nerul" becomes one landmark rather than three entities.

Table 12: Entity types

Entity	Meaning	Examples
POLE_ID	Pole identifier	SL-NER-021, pole no. 214
LOCATION	Area, sector or road name	Vashi, Sector 17, Palm Beach Road
LANDMARK	Named landmark, station or phrase after near/opposite/behind	SIES College, Vashi Station, the temple
DURATION	How long the fault has existed	since 3 days, for two weeks, 3 दिन से
FAULT	Fault phrase	flickering, not working, sparks, wire hanging, band hai, बंद

5.6 Severity Scoring

Severity is a transparent rule-weighted score rather than a learned one, so every point can be explained to a citizen or an auditor. The score starts from a base level for the fault class (outage 35, flicker 25, daytime burning 15, hazard 75, dim 18, vandalism 40, timer 30, multi-light outage 55) and adds points when evidence is found in the text (see the table below). The score is capped at 100 and mapped to a level: Critical (75 or more), High (55 to 74), Medium (30 to 54) and Low (below 30).

Table 13: Severity factors

Evidence	Points	Examples of cue words
Electrical-danger words (when the class is not already hazard)	+25	spark, exposed, live wire, shock, smoke
High-footfall or critical place	+10	school, college, hospital, station, bus stop, junction, market, temple
Public-safety concern	+12	women, girls, unsafe, dangerous, accident, theft
Vulnerable road users	+4	child, senior, elderly, patients, students
Night-time impact	+3	night, evening, after dark, रात, अंधार
Unresolved 3 to 6 days / 7 days or more	+8 / +14	since 5 days, for two weeks

The SLA deadline follows from the level: Critical 4 hours, High 12 hours, Medium 48 hours and Low 96 hours (configurable by the admin).

5.7 Geocoding and Lamp Matching

The location is resolved in order of trust. A GPS pin shared by the citizen is most reliable, followed by a pole ID found in the lamp register, a named landmark, a railway station, a sector within a zone, and finally a zone name alone (see the table below). The coordinates are then snapped to the nearest lamp using the haversine distance. Lamps that are already faulty are preferred, because a complaint is more likely to be about a lamp known to be failing.

Table 14: Location sources and confidence

Source of location	Confidence	Lamp search radius
GPS pin	0.99	120 m
Pole ID in text	0.98	exact lamp
Landmark in gazetteer	0.95	450 m
Station name	0.93	450 m
Sector within a zone	0.78	700 m
Zone name only	0.72	sent to review

5.8 Duplicate Detection

Before a ticket is created, the system looks for a live ticket for the same lamp. If the location was only approximate (confidence below 0.98), it also looks for a live ticket of the same fault class within 110 m. If a match is found, the new report is added to that ticket, its report count rises, and the citizen is told that the fault was already logged and that the priority has risen. Two reports with exact locations on different lamps stay separate, so that a row of failed lamps is not collapsed into one ticket.

5.9 Human-in-the-Loop Review

A ticket goes to the review queue instead of being processed automatically when no location can be extracted, when the location is too vague to choose a lamp, or when classifier confidence is below 55% (configurable). The operator confirms or corrects the fault type and chooses the lamp. Each correction is stored; pressing *Retrain* adds the corrections to the training data (repeated six times so that scarce examples carry weight) and refits the model. The interface then reports accuracy before and after.

5.10 Automatic Multilingual Reply

Every ticket creation returns a reply in the language detected. A confident ticket gets a confirmation with the reference number, priority and promised time; an unclear one asks for a landmark or a location pin; a duplicate says that the report was added to an existing ticket. The reply is stored with the ticket and, in a real deployment, would be sent through the same channel the complaint arrived on.

5.11 Worked Example

Table 15: Worked example through the pipeline

Step	Result for "Sparks coming from the pole near Vashi Station and a wire is hanging!"
Clean-up	No change needed.
Language	English.
Class	Electrical / pole hazard, probability above 0.95; words: pole, sparks, hanging from.
Entities	FAULT "Sparks", FAULT "wire is hanging", LANDMARK "Vashi Station".
Severity	Base level 75 for a hazard plus 10 for a high-footfall place (a railway station) gives 85, which is Critical.
Location	Landmark Vashi Station, confidence 0.95, snapped to lamp SL-VSH-046.

Step	Result for "Sparks coming from the pole near Vashi Station and a wire is hanging!"
Decision	Open ticket with a 4-hour SLA, auto-dispatched to the nearest HV-certified technician, red alert to the control room, reply to the citizen.

CHAPTER 6

Predictive Analytics and Optimisation

6.1 Hotspot Detection with DBSCAN

Two kinds of hotspot are computed. **Current hotspots** are found by running DBSCAN with the haversine metric on the coordinates of lamps that are currently faulty, with a radius of 170 m and a minimum of three lamps. Each cluster is reported with its centre, radius, number of lamps and a probable cause (a feeder pillar or MCB trip when four or more lamps fail together). **Repeat-failure hotspots** apply DBSCAN to 90 days of complaint locations with a radius of 75 m and at least eight points; the recommended action is to replace the driver or re-cable the stretch rather than repeat spot repairs. Clusters of four or more lamps also raise an alert.

6.2 Lamp Failure-Risk Model

A gradient-boosted classifier [6] (140 trees of depth 3, learning rate 0.08, subsampling 0.85) estimates the probability that each lamp fails within 14 days. The features are lamp age, faults in the last 90 days (computed live from real tickets), supply-voltage variation, rainfall in the last seven days, driver type (magnetic ballast, electronic, LED), burning hours and pole condition. Because no real failure records exist, the model is trained on 6,000 synthetic lamp records generated from a logistic relationship plus noise; 4,500 are used for training and 1,500 for testing. The administrator can drag a rainfall slider to see how the risk of all 508 lamps changes. For each lamp the dominant driver is found by replacing each feature in turn with the fleet median and measuring how far the risk falls. Selected lamps can be turned into preventive work orders that join the scheduling queue.

6.3 Weekly Fault Forecast

Weekly complaint counts for the last twelve weeks are forecast six weeks ahead with Holt's linear method with a damped trend [7], [23]. With level l , trend b , smoothing constants α and β and damping ϕ , the recursions are:

$$l_t = \alpha y_t + (1 - \alpha)(l_{t-1} + \phi b_{t-1})$$

$$b_t = \beta (l_t - l_{t-1}) + (1 - \beta) \phi b_{t-1}$$

$$\hat{y}_{t+h} = l_t + (\phi + \phi^2 + \dots + \phi^h) b_t$$

The parameters α , β and ϕ are chosen by grid search to minimise one-step-ahead squared error. The 80% prediction interval uses 1.28 times the standard deviation of past forecast errors, widened with the horizon. The mean absolute percentage error on the demonstration history is about 10%.

6.4 Priority Score

Ticket priority combines severity with time and citizen pressure:

$$\text{priority} = \text{severity} + \min(25, 0.6 \times \text{hours open}) + \min(25, 2.2 \times (12 - \text{hours to SLA})) + \min(15, 3 \times (\text{reports} - 1))$$

The third term applies only when fewer than 12 hours remain before the SLA deadline. The formula makes old, nearly-overdue and widely reported faults rise in the queue even when their severity is moderate.

6.5 Technician Scheduling

Each on-shift technician has a base location, a set of skills and a shift capacity of 480 minutes. Hazard jobs require an HV-certified electrician; multi-light outages require HV certification and a bucket truck. Travel speed is taken as 24 km/h and the on-site time is the typical repair time of the fault class. The optimiser works as follows:

1. Sort the open and unstarted tickets by priority, highest first.
2. For each ticket, try every eligible technician and every position in their route (cheapest insertion), rejecting positions that exceed the shift capacity.
3. Choose the insertion that increases the route cost least, where route cost = travel kilometres + $2.2 \times \Sigma$ (priority/100 $\times$ hours until the job is reached). The cost rewards short routes and early arrival at urgent jobs.
4. Improve each route with 2-opt (reverse a segment) [8] and or-opt (move one job) until no move lowers the cost.
5. Jobs that fit no shift are deferred to the next shift.

A **manual baseline** models a busy dispatcher: oldest complaint first, nearest depot, visit in arrival order, skills not checked. The interface shows both plans side by side. A critical ticket bypasses the batch: it is assigned immediately to the nearest eligible technician and placed first in their route.

6.6 Alerts Engine

Alerts are recalculated from live data rather than stored: critical faults, tickets past their SLA, tickets within two hours of SLA, high-priority tickets unassigned for three hours, hotspots of four or more lamps, zones with high predicted failure risk, and the size of the review queue. Alerts can be acknowledged and critical ones are pushed to every admin screen as soon as the ticket is created.

CHAPTER 7

Implementation

7.1 Tools and Technologies

Table 16: Tools used

Area	Tool	Use in the project
Language	Python 3.13	Backend, NLP, machine learning, optimisation, tests.
Web framework	Flask	REST API, sessions, static files, SSE.
Database	SQLite with WAL	Persistent storage, concurrent reads and writes.
Machine learning	scikit-learn [3], NumPy, SciPy	TF-IDF, logistic regression, gradient boosting, DBSCAN, metrics.
Maps	Leaflet, Esri street tiles	Lamp map, heat layer, hotspots, routes, pins.
Charts	Chart.js	Dashboard and analytics charts.
Front end	JavaScript modules, HTML, CSS	Single-page application without a build step.
Testing	pytest, Playwright	Automated tests; scripted screenshots and demo video.
Version control	Git, GitHub	https://github.com/bharath050305/prakash

7.2 Project Structure

Table 17: Project structure

Path	Contents
run.py	Starts the server (options: --reset, --host, --port).
backend/nlp.py	Normalisation, language ID, classifier, NER, severity, geocoding.
backend/training_data.py	Synthetic multilingual training and test complaints, hand-written check set.
backend/ml.py	Failure-risk model, forecast, DBSCAN hotspots.
backend/scheduler.py	Priority, route optimiser, manual baseline.
backend/services.py	Ingest pipeline, duplicates, ticket life cycle, alerts, dashboard statistics.
backend/app.py	REST API, roles, SSE, webhooks.
backend/db.py, seed.py, geo.py	Schema, demo data generator, city and lamp model.
frontend/	Single-page app: core helpers, map helpers, pages for citizen, technician and admin.

Path	Contents
tests/	Unit tests and an end-to-end API check.

The code base has about 5,200 lines of Python and JavaScript (3,300 in the backend). The source is available at <https://github.com/bharath050305/prakash>.

7.3 User Interface

The interface was designed for clarity and for presentation. It has a light and a dark theme, a responsive layout that works on phones, a navigation sidebar that depends on the role, live indicators and toast messages. The following figures show the main screens as captured from the running application.

Sign-in

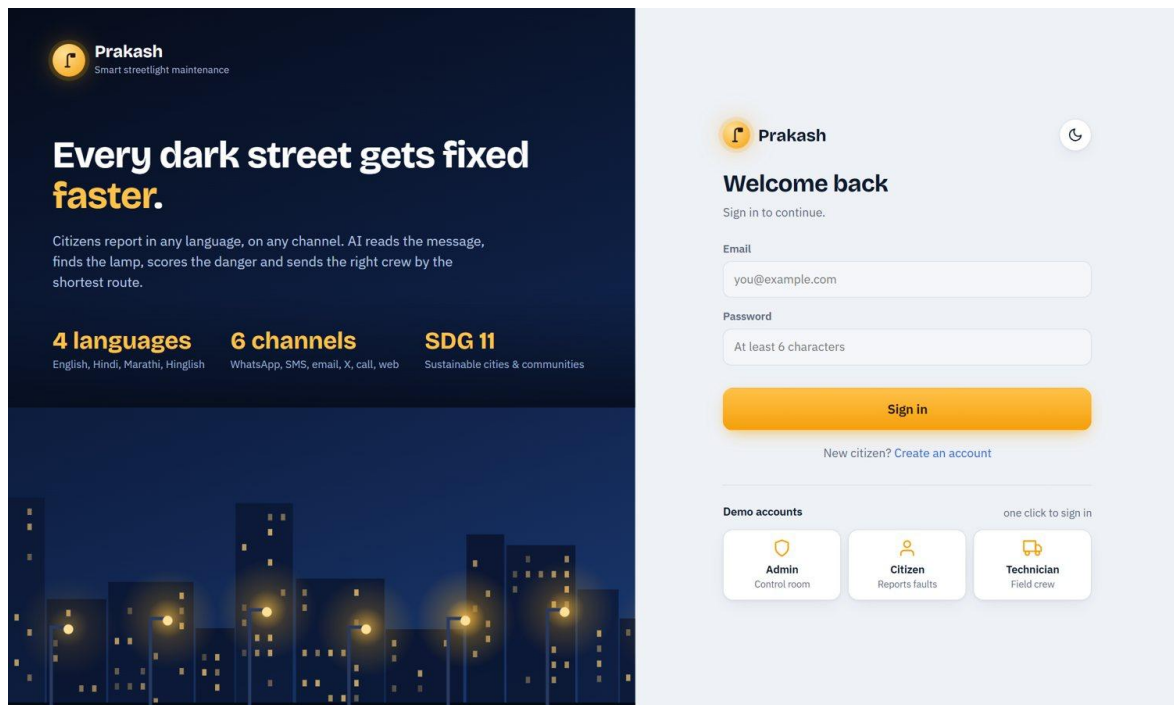


Figure 3: Sign-in page with one-click demonstration accounts for the three roles

Citizen screens

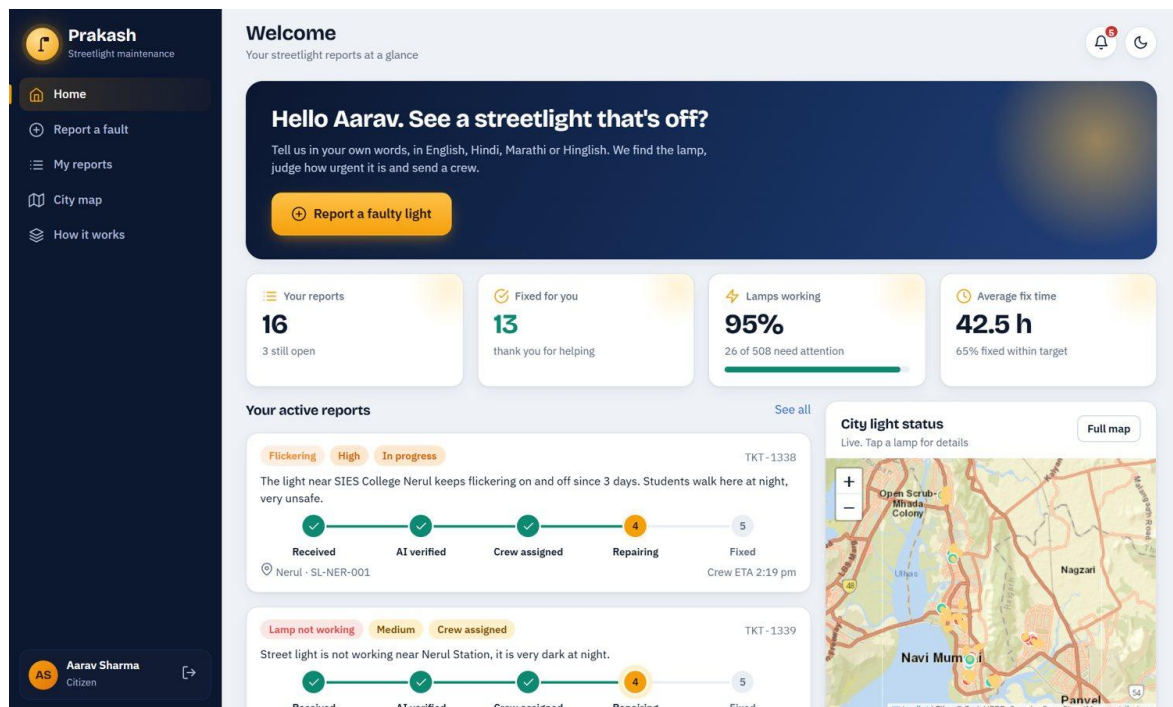


Figure 4: Citizen home: active reports with progress tracker and city lamp status

When a citizen types a complaint, the same pipeline used by the server runs on every pause in typing and shows the detected language, fault class, severity, extracted entities, matched lamp and expected fix time. Danger wording triggers a visible warning.

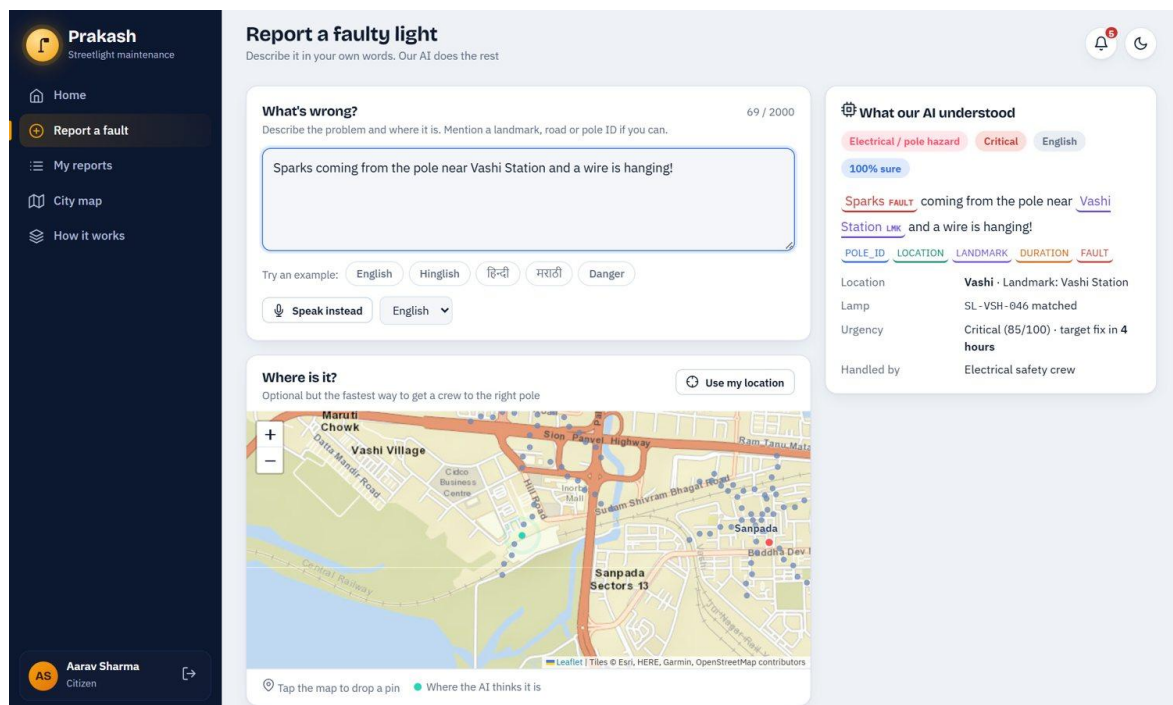


Figure 5: Report page with the live "what our AI understood" panel (English)

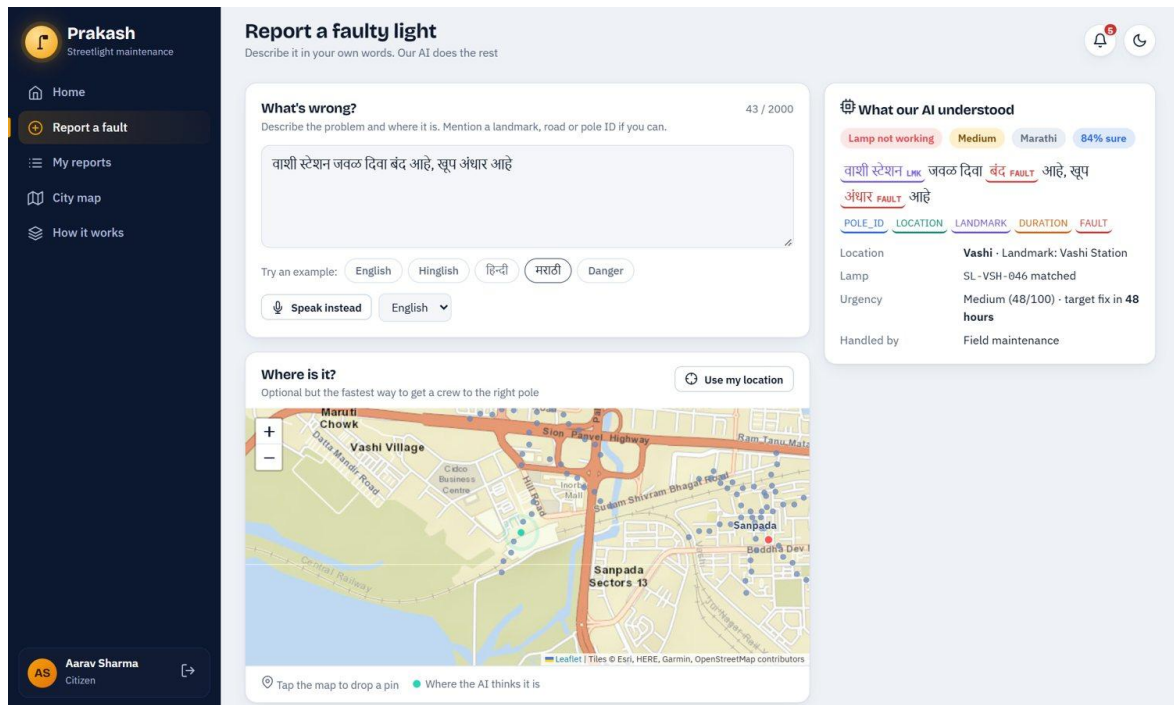


Figure 6: The same panel for a Hindi complaint, with the landmark and duplicate warning

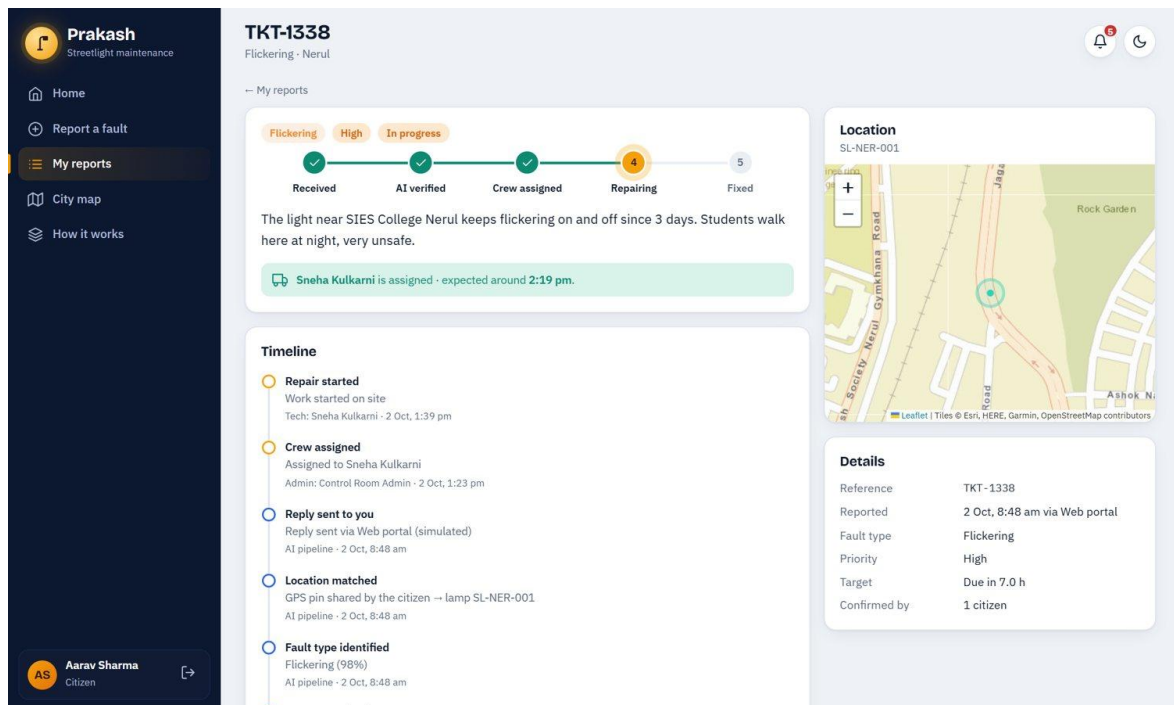


Figure 7: Citizen ticket tracking with progress stepper, timeline and location

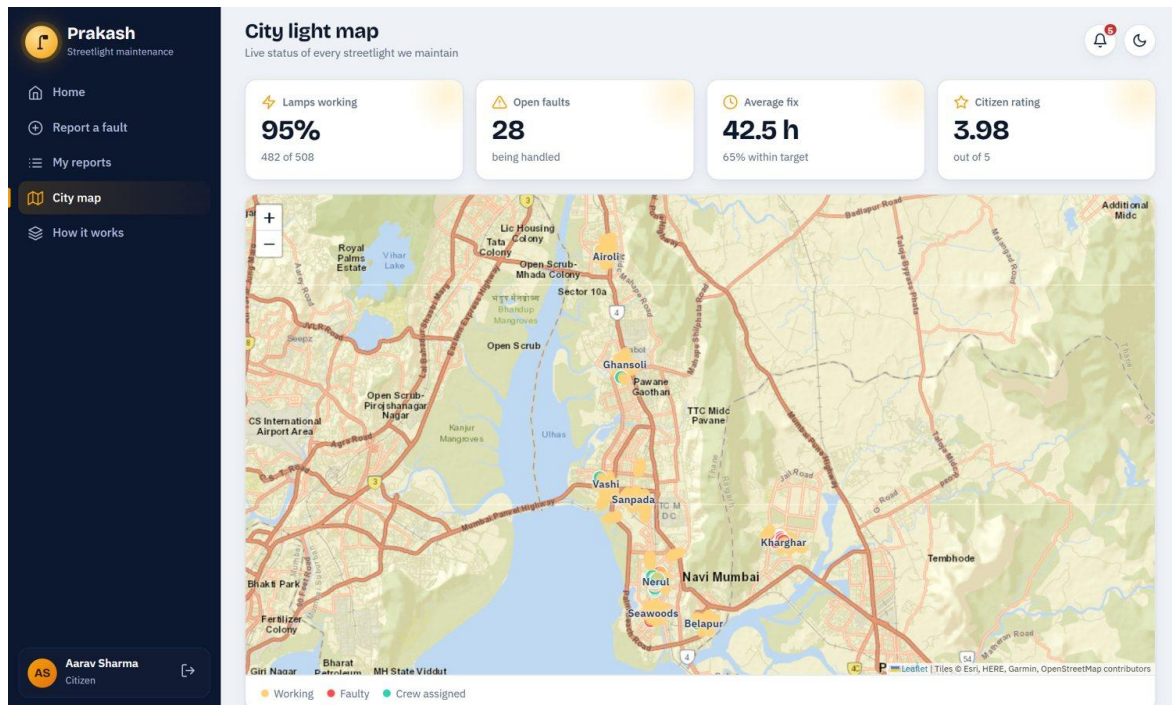


Figure 8: Public city map of lamp status

Admin screens

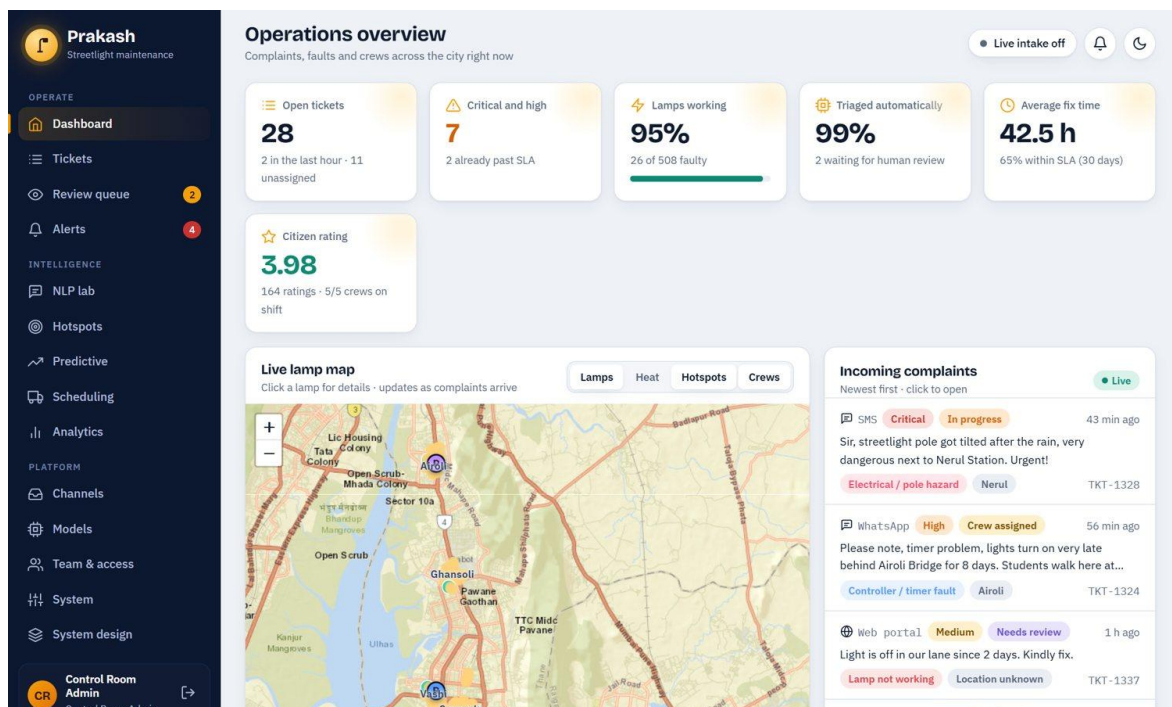


Figure 9: Admin dashboard with key indicators and live map

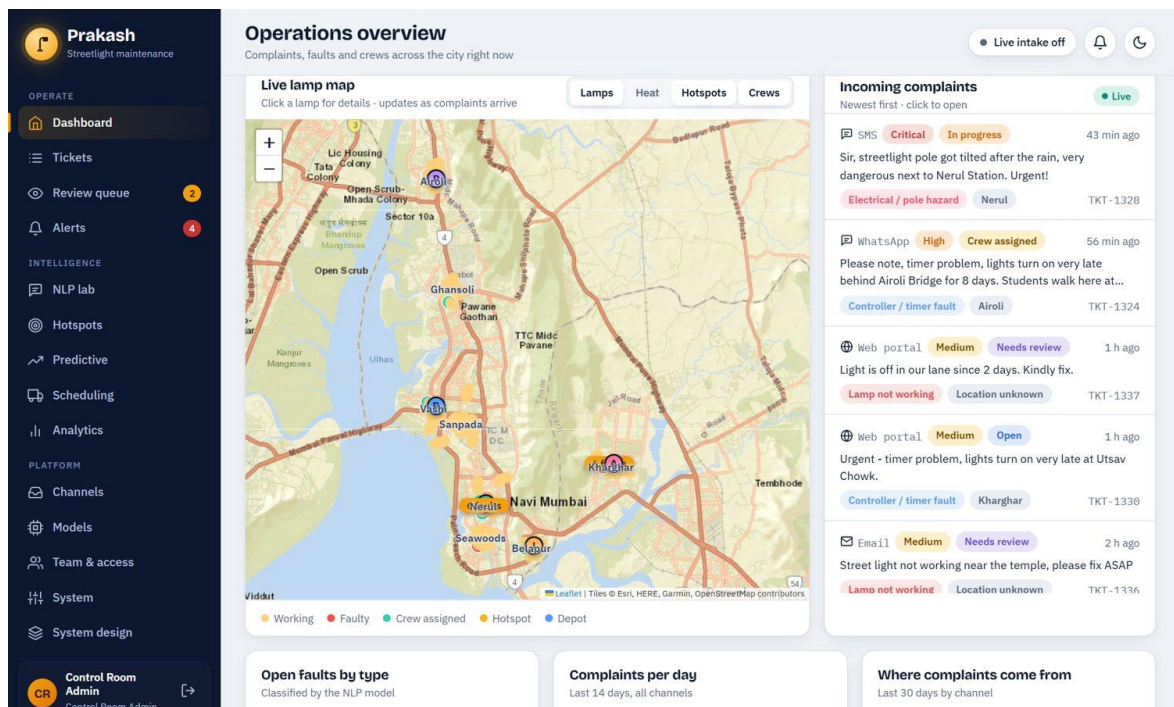


Figure 10: Live lamp map with hotspot clusters, and the live complaint feed

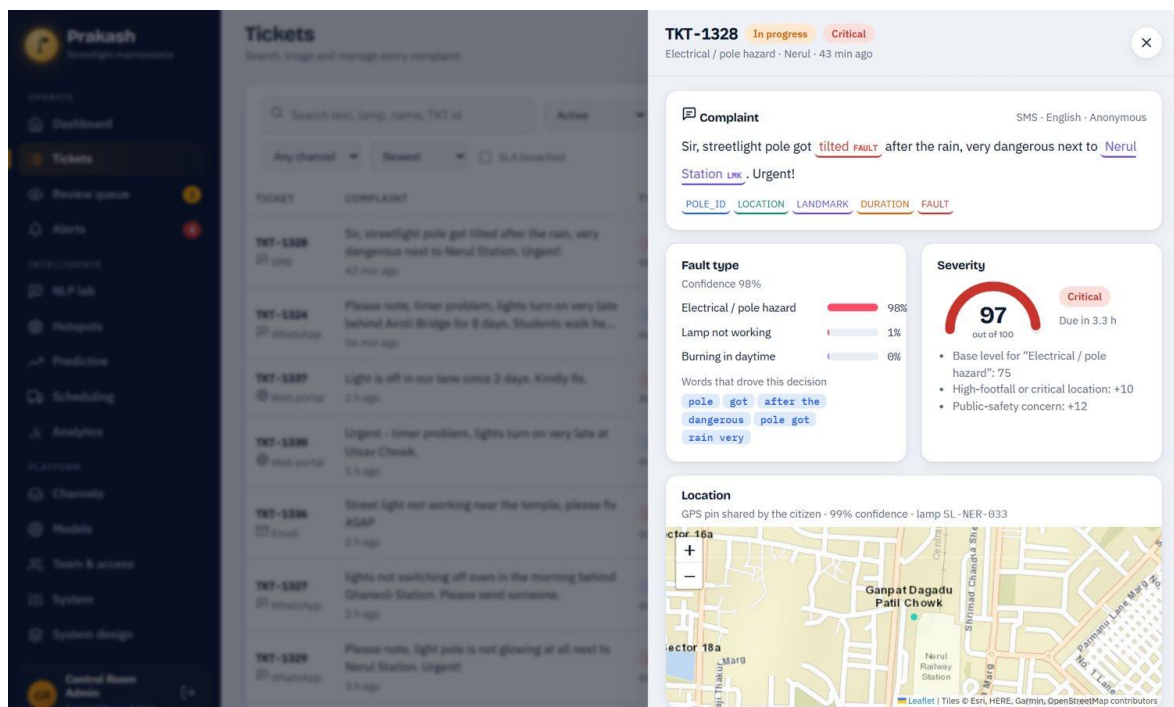


Figure 11: Ticket drawer: entities, class probabilities, severity explanation and location

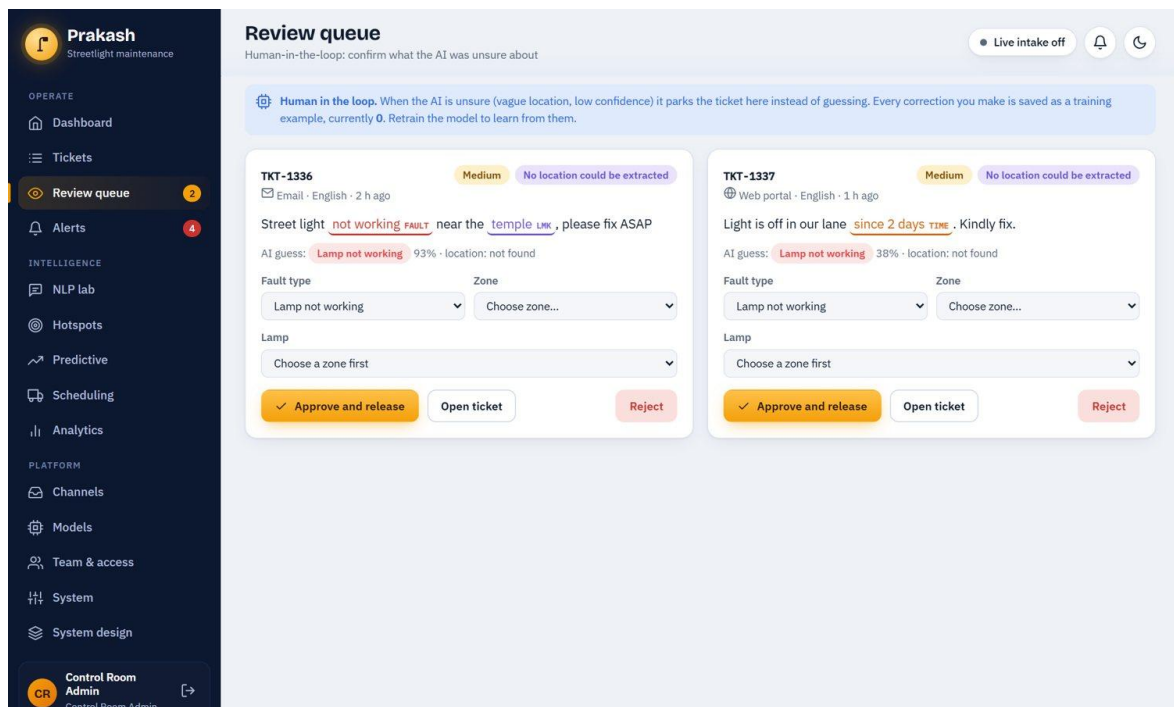


Figure 12: Review queue for uncertain tickets (human in the loop)

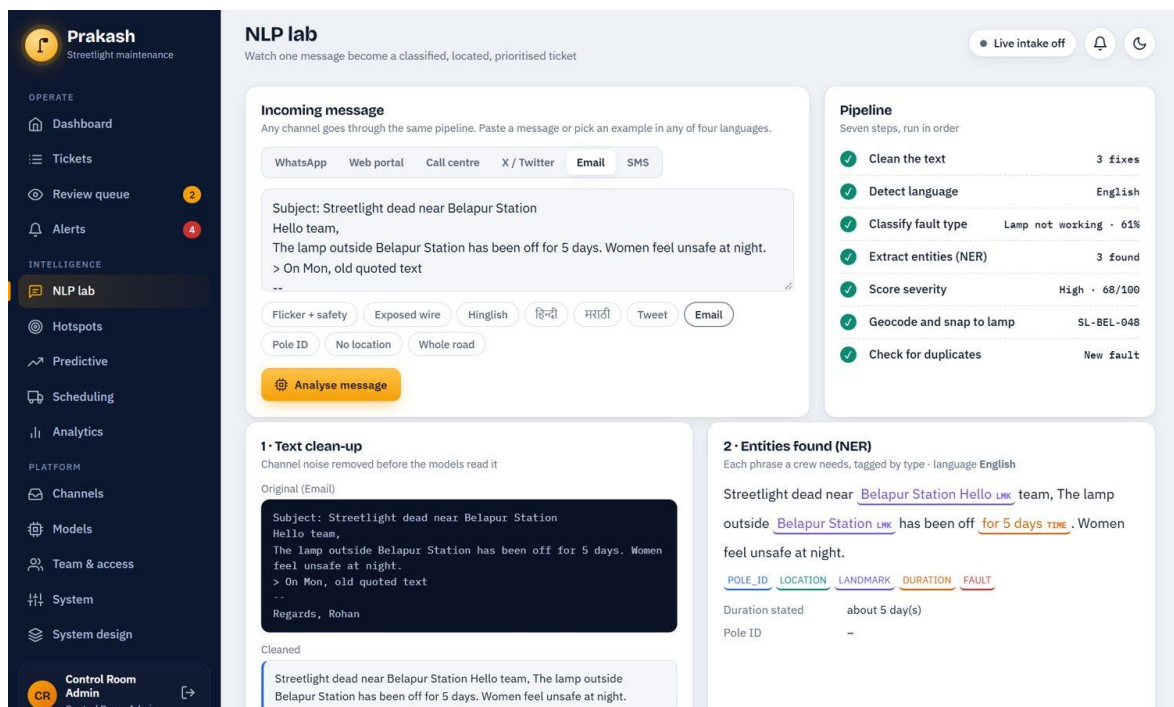


Figure 13: NLP lab: an email passes through seven pipeline steps

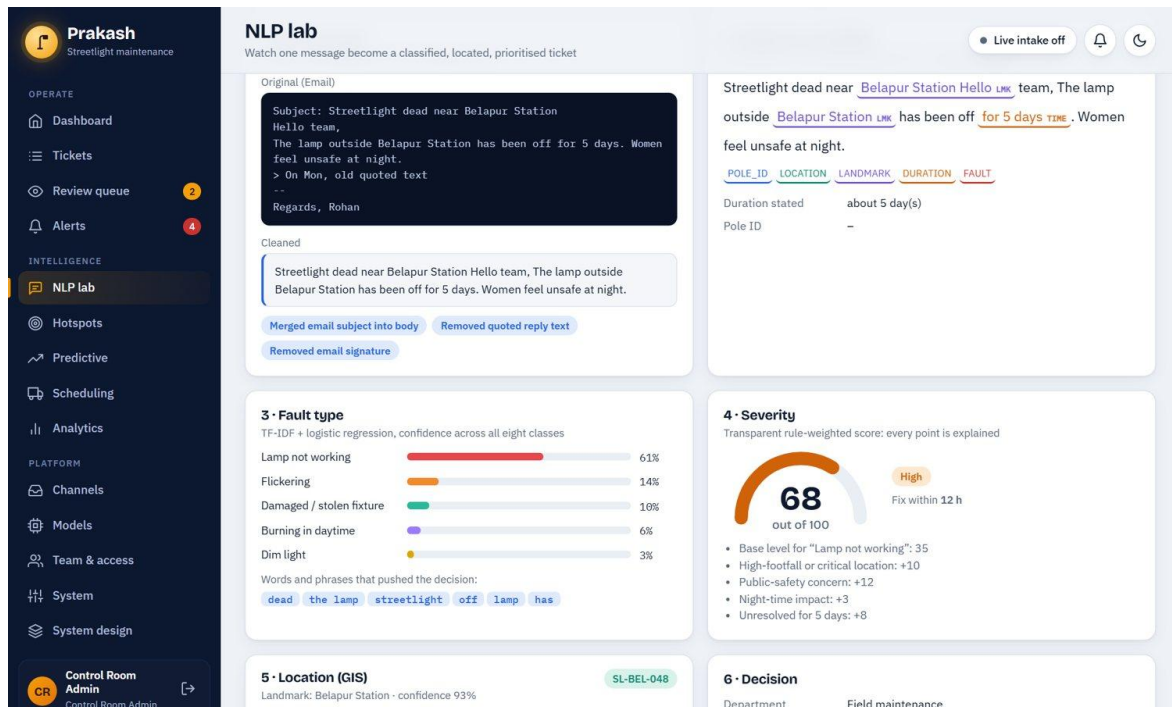


Figure 14: NLP lab: text clean-up and entity extraction for an email

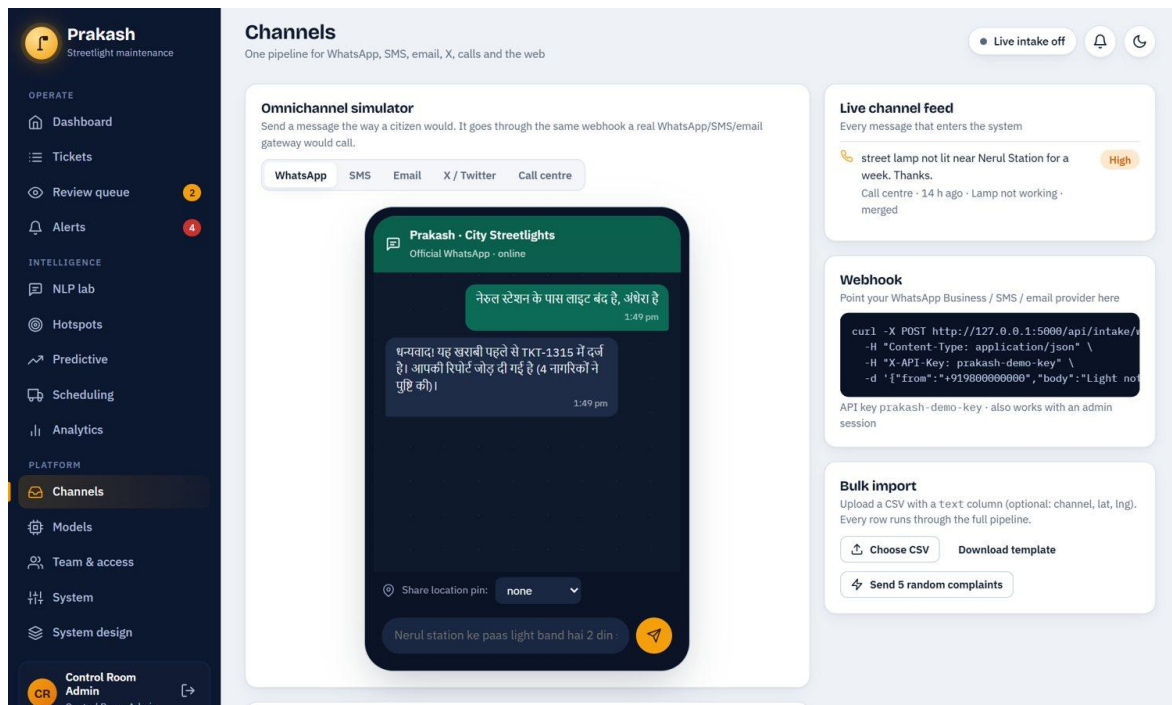


Figure 15: Omnichannel simulator: a Hindi WhatsApp message and the bot reply in Hindi

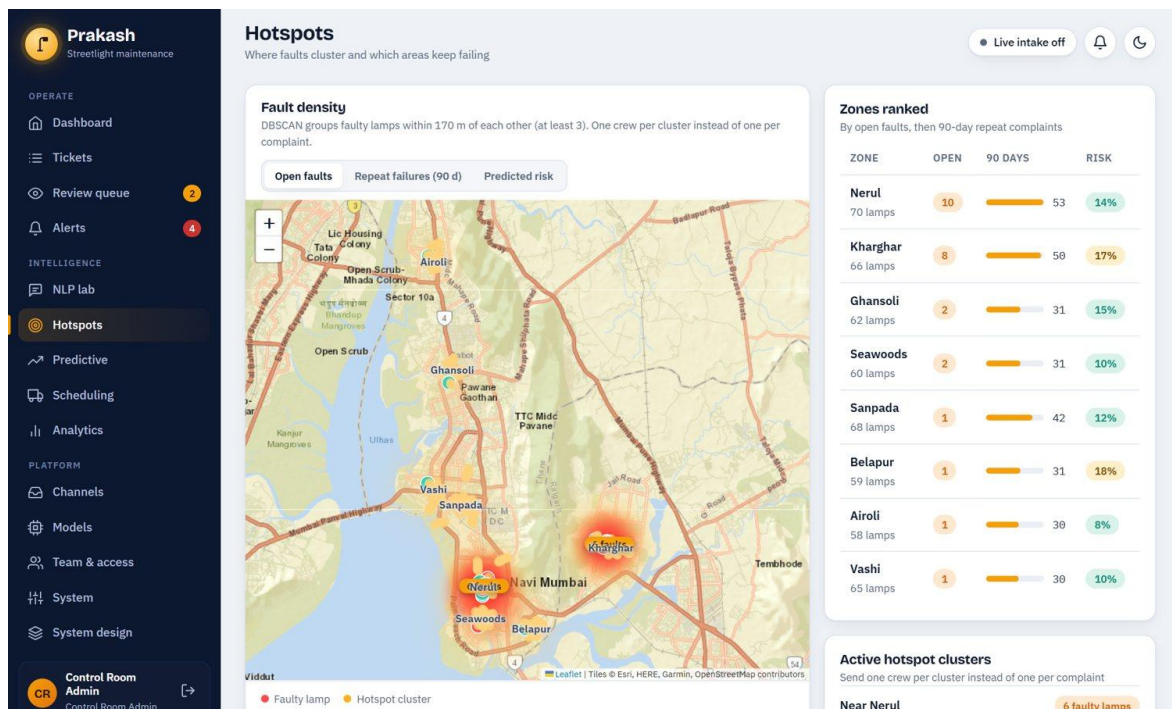


Figure 16: Hotspot analysis with ranked zones and cluster list

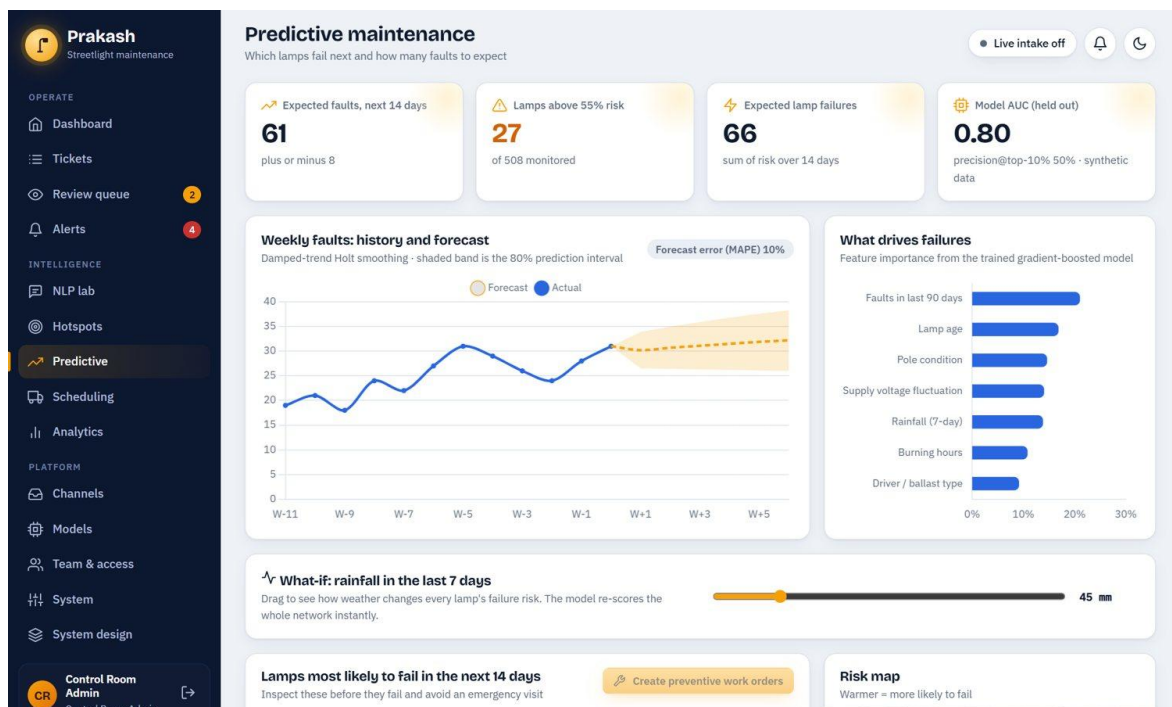


Figure 17: Predictive maintenance: forecast, feature importance and rainfall what-if

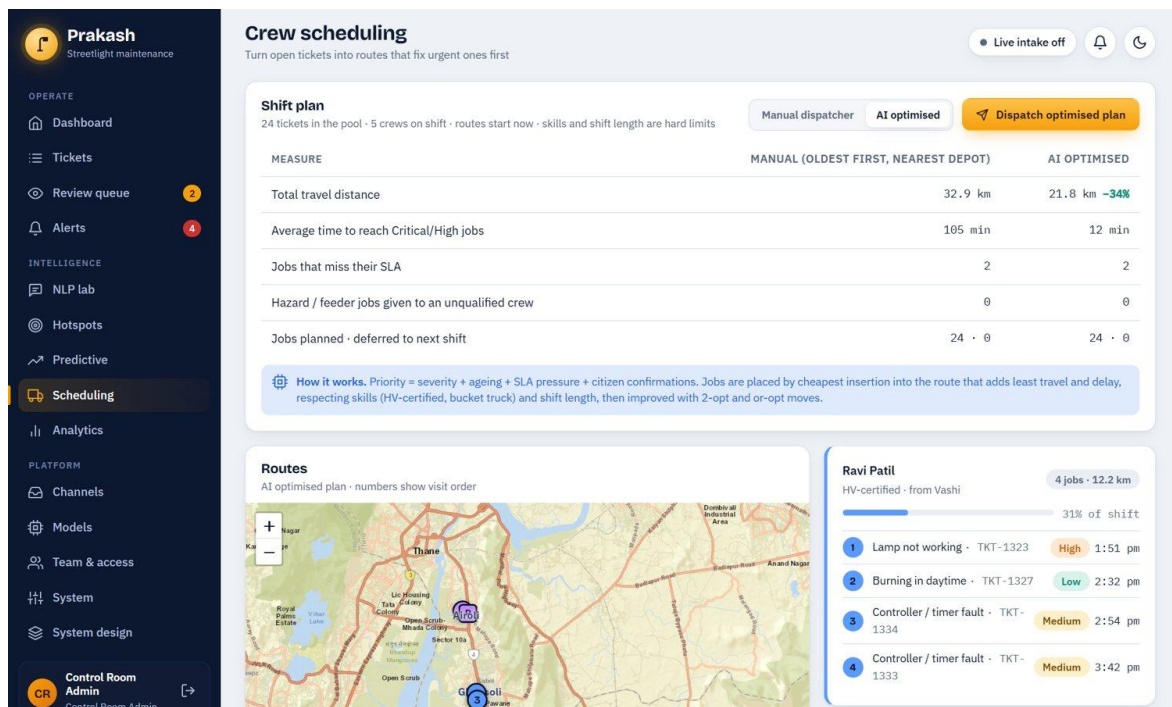


Figure 18: Crew scheduling: manual dispatcher compared with the optimiser

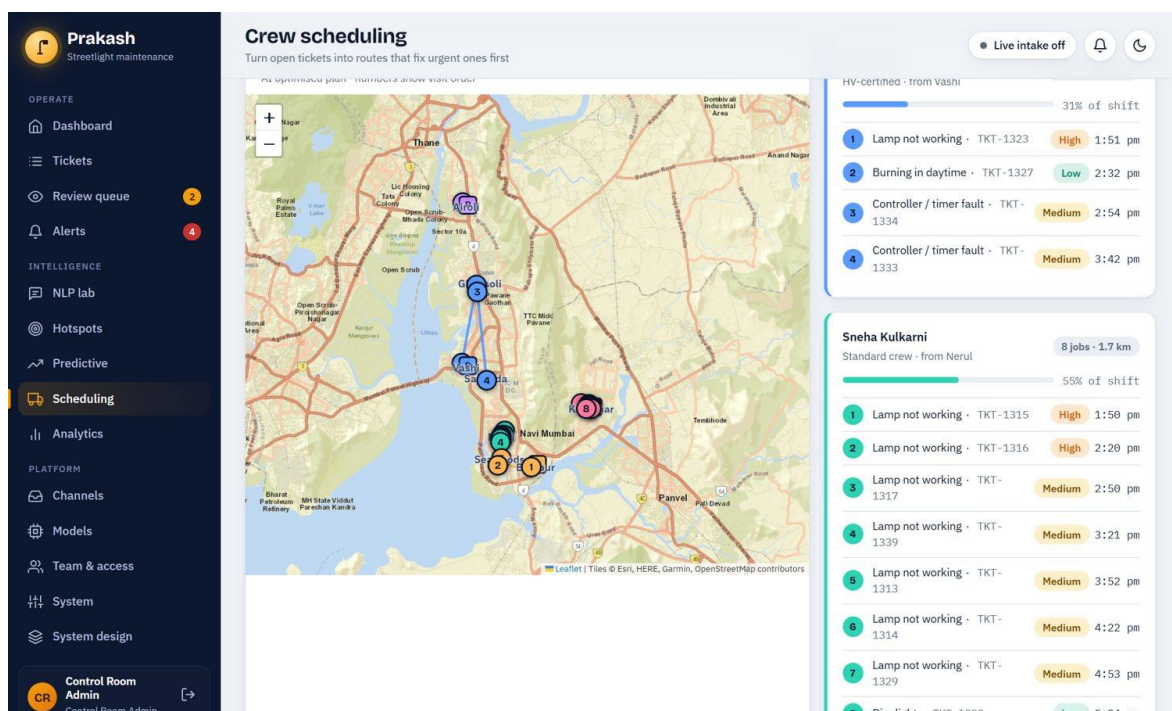


Figure 19: Optimised routes on the map with per-technician stop lists



Figure 20: Analytics: SLA compliance and throughput

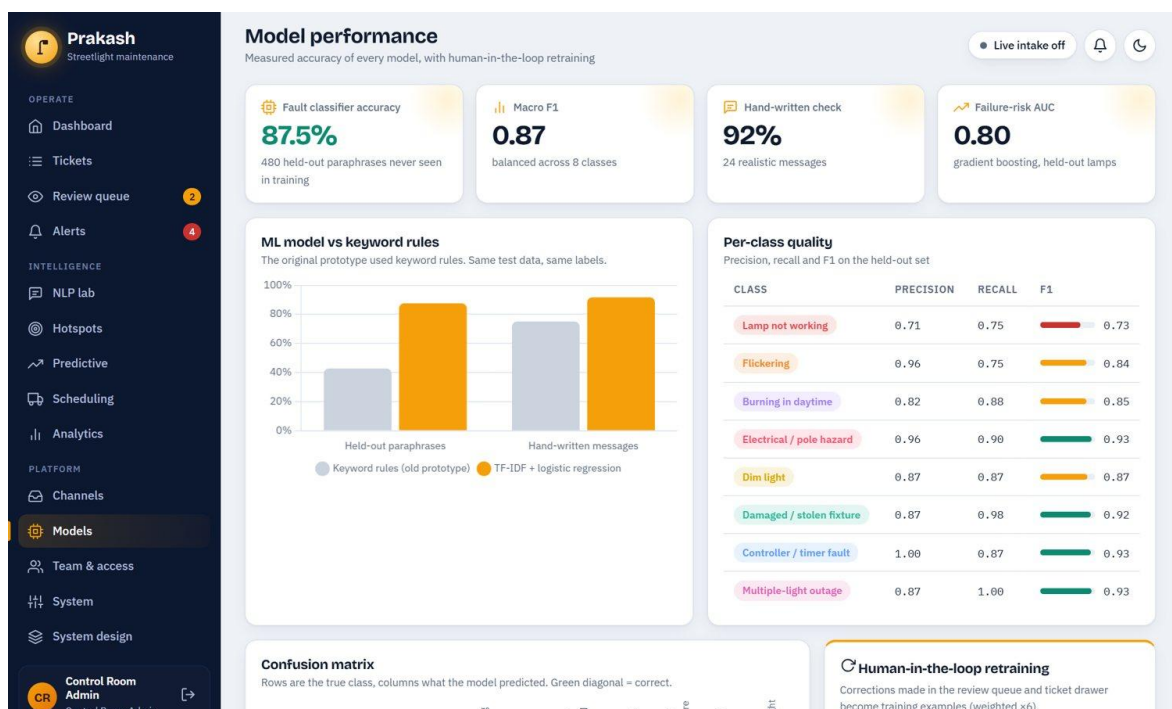


Figure 21: Model performance page: accuracy, baseline comparison and per-class quality

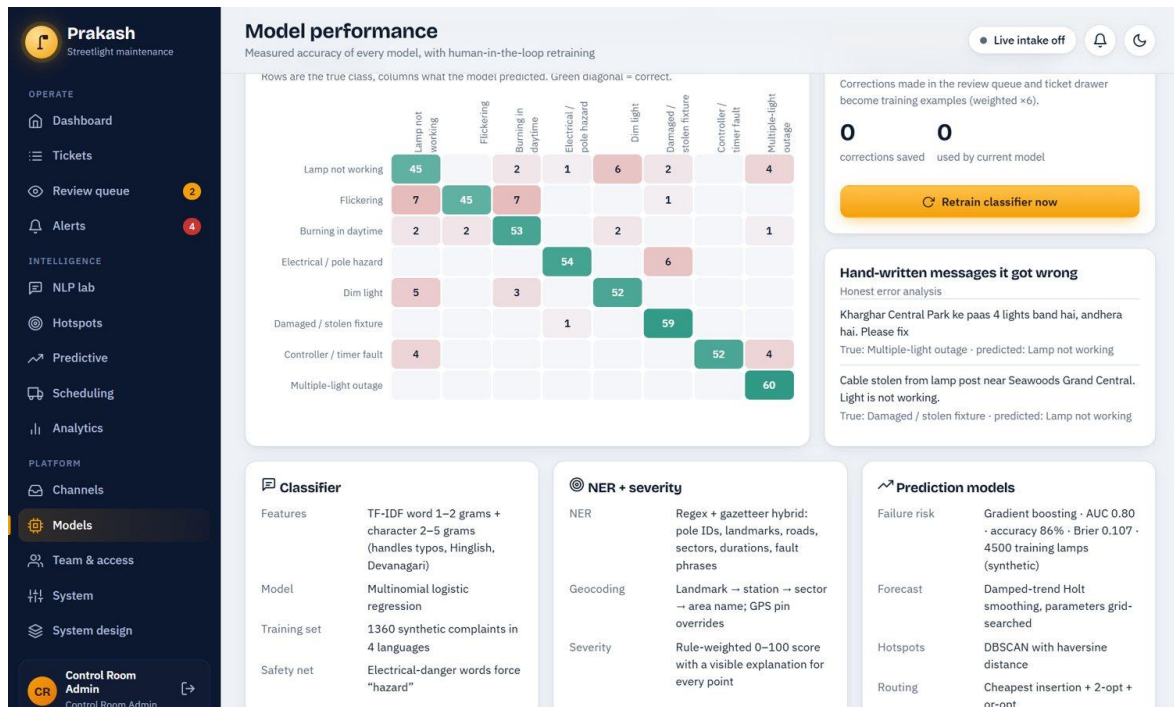


Figure 22: Confusion matrix and human-in-the-loop retraining

Technician screen

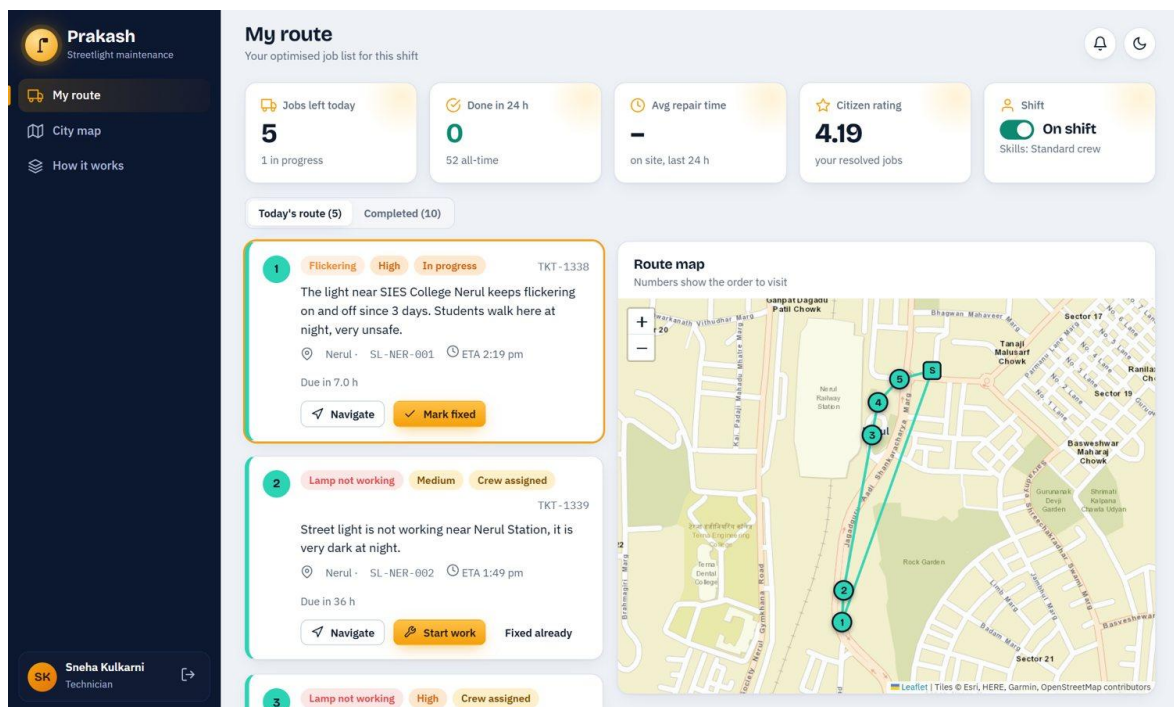


Figure 23: Technician route with numbered stops and job actions

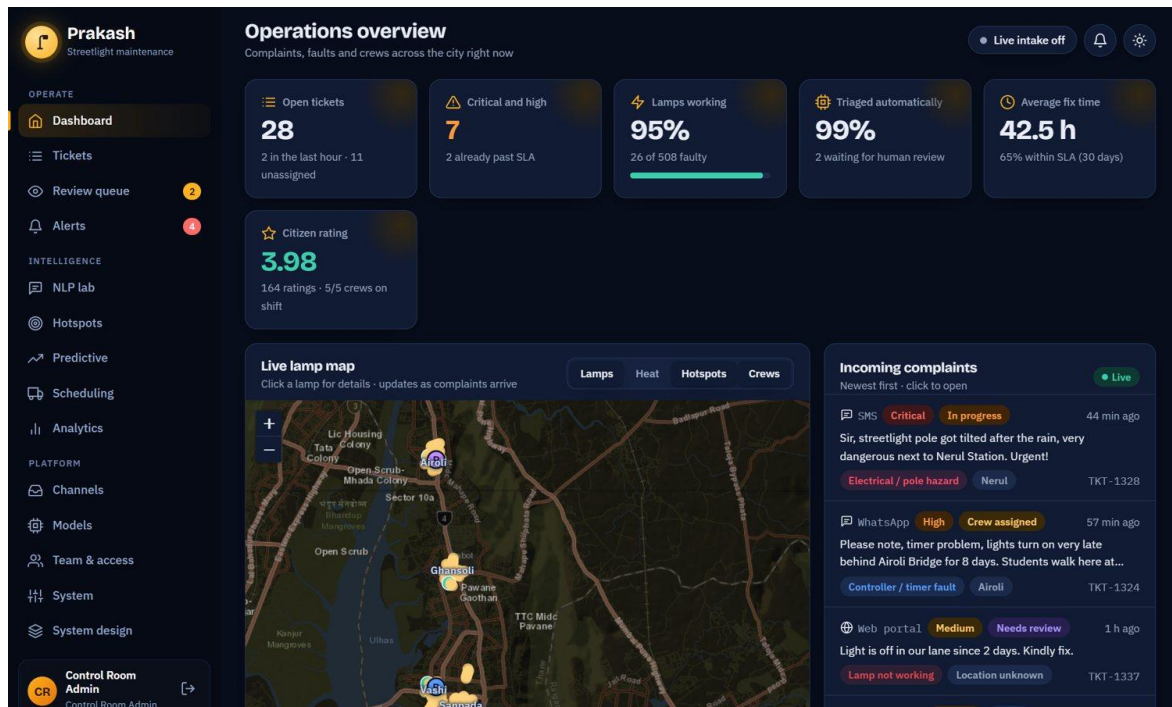


Figure 24: Dark theme of the admin dashboard

CHAPTER 8

Testing and Results

8.1 Test Strategy

Table 18: Automated tests

Level	Count	What is checked
Unit tests: NLP	8	Classifier metrics and margin over the baseline; language detection; electrical-danger safety rule; NER and geocoding; GPS override; channel clean-up; severity explanation adds up.
Unit tests: scheduler	5	Hazard jobs only to HV crews; cluster jobs need HV and bucket truck; capacity defers low-priority work; optimiser beats manual distance; off-shift crews ignored.
Service tests	7	Duplicate merge; review routing; exact-GPS reports on different lamps stay separate; lamp status through the life cycle; unqualified assignment rejected; risk and forecast; hotspots; review correction.
API tests	4	Role isolation; registration creates citizens only; citizens see only a public summary of others' tickets; webhook key check.
End-to-end API check	40 checks	Login for three roles, role isolation, citizen flow, admin flow, dispatch, webhooks, technician flow.

All 24 unit tests and all 40 end-to-end checks pass on the final code.

8.2 Classifier Results

Table 19: Fault classifier results

Measure	Result	Data
Accuracy (TF-IDF + logistic regression)	87.5%	480 held-out paraphrases
Macro-F1	0.875	same
Accuracy on hand-written messages	91.7% (22 of 24)	24 realistic messages
Keyword-rule baseline	42.7%	480 held-out paraphrases
Keyword-rule baseline	75.0%	24 hand-written messages

Table 20: Per-class precision, recall and F1 on the held-out set

Class	Precision	Recall	F1	Comment
outage	0.71	0.75	0.73	Most generic wording; confused with dim and flicker
flicker	0.96	0.75	0.84	Some synonyms unseen in training
dayburn	0.82	0.88	0.85	
hazard	0.96	0.90	0.93	Safety rule helps recall
dim	0.87	0.87	0.87	
vandal	0.87	0.98	0.92	
timer	1.00	0.87	0.93	
cluster	0.87	1.00	0.93	

The weakest class is *outage* (F1 0.73) because "lamp not working" is the most generic description and other faults are sometimes described in similar words. The confusion matrix shown in Chapter 7 shows that errors are mainly between outage, dim and flickering, which are all minor faults, whereas the most dangerous class (hazard) is recognised with high precision. The hand-written check also exposes two errors, both involving descriptions that mix two faults (for example a stolen cable and "light is not working").

8.3 Effect of the Safety Rule and Review Threshold

The safety rule guarantees that danger words are routed as hazards. The 55% confidence threshold sends vague messages to a human. In the demonstration data this affects a small fraction of tickets (about 99% are triaged automatically with two or three waiting for review), which keeps operator effort low while limiting wrong automatic decisions.

8.4 Failure-Risk Model Results

Table 21: Failure-risk model results

Measure	Result	Data
AUC	0.80	1,500 held-out synthetic lamps
Accuracy	85.6%	same (failure base rate 15.4%)
Brier score	0.107	same
Precision in the top 10% of risk	0.50	same

Table 22: Feature importance of the failure-risk model

Feature	Importance
Faults in the last 90 days	0.21
Lamp age	0.17
Pole condition	0.15
Supply-voltage variation	0.14
Rainfall (7 days)	0.14
Burning hours	0.11
Driver / ballast type	0.09

The model recovers the relationship used to generate the data, as expected; the AUC therefore shows that the pipeline and features work, not that the predictions are valid for real lamps. The forecast has a mean absolute percentage error of about 10% on the demonstration history.

8.5 Scheduling Results

On the demonstration data (24 open tickets, five technicians) the optimiser was compared with the manual baseline:

Table 23: Optimiser compared with the manual baseline

Measure	Manual dispatcher	Optimiser
Total travel distance	32.9 km	21.8 km (about 34% shorter)
Average time to reach Critical and High jobs	105 min	12 min
Hazard or feeder jobs given to an unqualified crew	0 in this run (2 in other runs)	0
Jobs planned / deferred	24 / 0	24 / 0

The distance advantage depends on the situation. With extra hazard jobs that only HV-certified crews may take, the optimiser may travel as far as the baseline, but it still reaches urgent jobs much sooner and never violates skills. The automated end-to-end check therefore asserts a competitive distance and a shorter time to urgent jobs, not a fixed saving.

8.6 Performance

Table 24: Measured response times on a laptop (local server)

Operation	Mean time	Slowest of runs
Live AI preview (/api/analyze)	29 ms	32 ms

Operation	Mean time	Slowest of runs
Dashboard numbers (/api/admin/overview)	15 ms	18 ms
Ticket list page (25 rows)	17 ms	21 ms
Both schedule plans (24 tickets, 5 technicians)	37 ms	41 ms
Failure-risk scores for 508 lamps with predictions page	51 ms	57 ms
Map data with risk and hotspots	63 ms	72 ms
Cold start (training both models and generating data)	about 7 s	

8.7 Threats to Validity and Limitations

- **Synthetic data.** Complaint text, lamp register, history and ratings were generated, so accuracy numbers describe the pipeline, not real-world performance. Real messages are longer, noisier and more varied.
- **Template-based test set.** The held-out set uses different phrasings but the same generator style; the hand-written set is small (24).
- **Model simplicity.** TF-IDF with logistic regression cannot capture deep context or negation reliably. A transformer would likely perform better with real data.
- **Rule-based NER and severity.** They are transparent but limited to the gazetteer and rules written for this city.
- **Simulated channels.** The webhooks are real endpoints, but no WhatsApp Business or SMS provider was connected.
- **Routing.** Travel uses straight-line distance at a constant speed, not road-network times.
- **Deployment.** The demonstration uses a development web server and SQLite.

CHAPTER 9

Social Impact, Ethics and Sustainability

9.1 Expected Impact

- **Faster repair of dangerous faults:** hazards are classified within milliseconds, flagged as critical, and sent to the nearest certified crew, instead of waiting in a queue.
- **Less wasted effort:** merging duplicates and planning routes reduces repeat visits and kilometres driven.
- **More inclusive reporting:** citizens can write in Hindi, Marathi or Hinglish, and can speak instead of typing.
- **Transparency:** citizens see progress and rate repairs; administrators see SLA compliance and where infrastructure fails.
- **Prevention:** failure-risk prediction and hotspot analysis support planned replacement before failures happen.

9.2 Ethical Considerations

- **Privacy and data protection.** Only the data needed to handle a complaint is stored (name, optional phone, complaint, location). Access is limited by role. A real deployment must comply with India's Digital Personal Data Protection Act, 2023 [14], including consent, purpose limitation and retention limits.
- **Human oversight.** The system never silently guesses: uncertain tickets go to a human, every automated decision is explained, and an administrator can override any classification.
- **Fairness across areas and languages.** Severity depends on the content of a complaint, not on which neighbourhood it came from, but a model trained mostly on one language or area could serve others worse. Per-language and per-zone evaluation on real data is needed before deployment, and the language mix is already reported on the analytics page.
- **Safety first.** The deterministic danger rule favours false alarms over missed hazards.
- **Accountability.** Ticket timelines and an audit log record who did what and when.

9.3 Sustainability

Detecting lamps that burn during the day avoids wasted electricity. Predictive replacement of ageing ballast-type drivers supports the move to efficient LED fixtures. Shorter crew routes reduce fuel use and emissions. The software itself is lightweight: it trains and runs on a laptop, which keeps its own energy footprint small.

CHAPTER 10

Mapping to Course Outcomes, Programme Outcomes and Knowledge Profiles

This chapter relates the project to the outcomes of the NLP course and to the programme outcomes (PO) and knowledge profiles (WK) used for Complex Engineering Problems. Course-outcome numbers follow the order of topics in the course.

10.1 Course Outcome Mapping

Table 25: Course outcome mapping

CO	Outcome (summary)	Where it is demonstrated
CO1	Apply text pre-processing and feature representation techniques (tokenisation, normalisation, TF-IDF, n-grams).	Chapter 5.2 to 5.4: channel clean-up, word and character n-gram TF-IDF.
CO2	Apply language-modelling and classification techniques to text.	Chapter 5.4: logistic regression, probabilities, evaluation with precision, recall, F1.
CO3	Apply sequence labelling and information extraction (NER).	Chapter 5.5 and 5.7: entity types, gazetteer, geocoding.
CO4	Handle multilingual and code-mixed text, and compare approaches.	Chapter 5.3: Hindi, Marathi, Hinglish; comparison with the keyword baseline.
CO5	Build, evaluate and present an NLP application addressing a real-world problem.	Chapters 4, 7, 8: full system, testing and results, demonstration.

10.2 Programme Outcome Mapping

Table 26: Programme outcome mapping

PO	Programme outcome	Evidence	Level
PO1	Engineering knowledge	Applied mathematics (probability, TF-IDF, haversine, optimisation) and computing fundamentals.	High
PO2	Problem analysis	Decomposed the problem statement into nine objectives and a five-stage pipeline (Chapters 1, 3).	High
PO3	Design and development of solutions	Designed architecture, data model, algorithms and interfaces (Chapters 4 to 7).	High
PO4	Investigation of complex problems	Experiments on held-out data, baseline comparison, error analysis (Chapter 8).	High

PO	Programme outcome	Evidence	Level
PO5	Modern tool usage	scikit-learn, Flask, SQLite, Leaflet, Chart.js, Git, automated tests.	High
PO6	The engineer and society	Public safety, inclusive reporting, SDG 11 (Chapters 1.5, 9).	High
PO7	Environment and sustainability	Energy waste detection, shorter routes, preventive replacement (Chapter 9.3).	Medium
PO8	Ethics	Privacy, human oversight, transparency, safety-first design (Chapter 9.2).	Medium
PO9	Individual and team work	Three-member team with shared repository and review of each other's work.	Medium
PO10	Communication	This report, presentation, narrated demo video, README and demo guide.	High
PO11	Project management and finance	Phased plan from prototype to full system, version control, tests; no paid services needed.	Medium
PO12	Life-long learning	Self-learning of NLP, geospatial clustering, forecasting and web engineering beyond the syllabus.	Medium

10.3 Knowledge Profile (WK) Mapping

Table 27: Knowledge profile mapping

WK	Knowledge profile	Evidence
WK2	Mathematics, statistics and computer science	Probability and logistic regression, TF-IDF, DBSCAN, exponential smoothing, heuristic optimisation.
WK3	Engineering fundamentals	Algorithms and data structures, databases, networking (REST, SSE).
WK4	Specialist engineering knowledge	Natural language processing, machine learning, GIS and operations research.
WK5	Engineering design	System architecture, data model, API and user-interface design.
WK6	Engineering practice	Software engineering practice: testing, version control, security, deployment.
WK7	Engineering in society	Municipal service delivery, public safety, SDG 11, citizen participation.

WK	Knowledge profile	Evidence
WK8	Research literature	Use of published methods for TF-IDF, NER, DBSCAN, gradient boosting, Holt smoothing and routing (References).
WK9	Ethics, inclusive behaviour and conduct	Privacy, fairness across languages, human oversight, honest reporting of limitations.

CHAPTER 11

Conclusion and Future Scope

11.1 Conclusion

This project set out to show that one pipeline can turn raw, multilingual streetlight complaints into classified, located, prioritised and scheduled repairs, while sending uncertain cases to a human. Prakash does this end to end. It reads complaints from six channels in English, Hindi, Marathi and Hinglish; classifies the fault with 87.5% accuracy on held-out wording; extracts entities and resolves them to a specific lamp; explains a severity score; merges duplicates; finds hotspots; predicts failures; and builds skill-aware routes that reach urgent jobs about nine times sooner than a manual baseline in our demonstration. It does this in a role-based web application with live updates, alerts, a citizen tracking portal and a technician route view, and it is tested by 24 automated tests and 40 end-to-end checks.

The work also made limits clear. Without real labelled data, accuracy figures describe the pipeline on synthetic complaints and must not be read as real-world performance. The value of the project is therefore the complete, explainable and testable system, and the human-in-the-loop mechanism that lets it learn from real operators once deployed.

11.2 Future Scope

- Collect and label 1,500 to 2,000 real complaints in English, Hinglish, Hindi and Marathi, and report per-language accuracy, F1 and confusion matrices.
- Replace or complement the classifier with DistilBERT [11] or MuRIL [12], and a trained NER model (for example spaCy) with entity-level F1.
- Train the failure-risk model on real maintenance records and weather data; compare gradient boosting with other tabular models.
- Use road-network travel times and an exact or metaheuristic VRP solver such as Google OR-Tools for scheduling.
- Connect a WhatsApp Business account and an SMS gateway to the existing webhooks; integrate with the municipal ticketing system and lamp register.
- Add image analysis of citizen photos (for example to confirm a hanging wire) and live crew GPS.
- Move to PostgreSQL with PostGIS, deploy behind HTTPS with a production server, and add a mobile app for technicians.

References

- [1] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," General Assembly resolution A/RES/70/1, 2015.
- [2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513-523, 1988.
- [3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [4] D. Nadeau and S. Sekine, "A survey of named entity recognition and classification," *Linguisticae Investigationes*, vol. 30, no. 1, pp. 3-26, 2007.
- [5] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, pp. 226-231, 1996.
- [6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189-1232, 2001.
- [7] E. S. Gardner Jr. and E. McKenzie, "Forecasting trends in time series," *Management Science*, vol. 31, no. 10, pp. 1237-1246, 1985.
- [8] G. A. Croes, "A method for solving traveling-salesman problems," *Operations Research*, vol. 6, no. 6, pp. 791-812, 1958.
- [9] P. Toth and D. Vigo (eds.), *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: SIAM, 2014.
- [10] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.
- [11] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.
- [12] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," *arXiv:2103.10730*, 2021.
- [13] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [14] Government of India, "The Digital Personal Data Protection Act, 2023."
- [15] Flask documentation, Pallets Projects. <https://flask.palletsprojects.com>
- [16] Leaflet: an open-source JavaScript library for interactive maps. <https://leafletjs.com>
- [17] Chart.js documentation. <https://www.chartjs.org>
- [18] City of New York, "311 Service Requests from 2010 to Present," NYC Open Data. <https://opendata.cityofnewyork.us>
- [19] City of Chicago, "311 Service Requests," Chicago Data Portal. <https://data.cityofchicago.org>
- [20] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York: Springer, 2009.

- [21] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft), 2024.
- [22] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.
- [23] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," International Journal of Forecasting, vol. 20, no. 1, pp. 5-10, 2004.
- [24] Prakash source code, GitHub repository. <https://github.com/bharath050305/prakash>

Appendix A: Running the Project

Source code: <https://github.com/bharath050305/prakash>

Requirements: Python 3.10 or later. Open a terminal in the project folder and run:

```
pip install -r requirements.txt
python run.py
```

Then open <http://localhost:5000>. The first start trains the models and generates demonstration data (about 7 seconds). The sign-in page offers one-click demonstration accounts for the Admin, Citizen and Technician roles.

Table 28: Commands

Command	Effect
<code>python run.py --reset</code>	Wipe the database and regenerate fresh demonstration data.
<code>python run.py --host 0.0.0.0</code>	Allow phones on the same Wi-Fi to open the app.
<code>python run.py --port 8080</code>	Use another port.
<code>python -m pytest tests -q</code>	Run the 24 automated tests.
<code>python tests/smoke_api.py</code>	Run the 40-check end-to-end API test against a running server.

Appendix B: REST API Summary

Table 29: REST API summary

Method	Endpoint	Purpose
POST	/api/auth/login, /register, /logout	Authentication
GET	/api/auth/me, /api/meta	Session and reference data
POST	/api/analyze	Run the NLP pipeline without creating a ticket
POST	/api/tickets	Create a ticket (citizen)
GET	/api/my/tickets, /api/tickets/{id}	Citizen ticket list and detail
POST	/api/tickets/{id}/confirm, /rate	Confirm an existing fault; rate a repair
GET	/api/nearby, /api/city/map	Nearby reports; public lamp map
GET	/api/notifications; POST /read	In-app notifications
GET	/api/stream	Server-sent events
GET	/api/tech/jobs; POST /shift, /jobs/{id}/start, /complete	Technician functions
GET	/api/admin/overview, /map, /tickets, /hotspots, /predictions, /schedule, /analytics, /alerts, /model	Admin dashboards and analytics
POST	/api/admin/tickets/{id}/assign, /status, /review, /note	Ticket management
POST	/api/admin/schedule/dispatch, /preventive, /model/retrain, /import, /reset	Dispatch, preventive work orders, retraining, bulk import, reset
GET/PUT	/api/admin/settings; GET /export/tickets.csv, /audit	Settings, export and audit log
POST	/api/intake/{whatsapp, sms, email, twitter, call, web}	Channel webhooks (API key or admin session)

Appendix C: Sample Complaints and Outputs

Table 30: Sample complaints and system outputs

Complaint (channel)	Language / class	Severity and location
"The light near SIES College Nerul keeps flickering on and off since 3 days. Students walk here at night, very unsafe." (WhatsApp)	English / flicker	High (62); landmark SIES College; duration 3 days
"Caller reports sparks and a wire hanging from the pole near Vashi Station!" (Call centre)	English / hazard (safety rule)	Critical (85); landmark Vashi Station
"Kharghar Central Park ke paas 4 lights band hai, andhera hai." (WhatsApp)	Hinglish / outage	Medium; landmark Kharghar Central Park
"नेरुल स्टेशन के पास स्ट्रीट लाइट बंद है, अंधेरा है" (WhatsApp)	Hindi / outage	Medium (48); Nerul Station
"SL-NER-021 not glowing near D Y Patil Stadium" (Web)	English / outage	Medium; pole ID SL-NER-021 matched
"Street light not working near the temple, please fix ASAP" (Email)	English / outage	No location: sent to human review